

2024 年度未踏 IT 人材発掘・育成事業

Capability-Based Microkernel による セキュアなユーザレベルメモリ管理システム 成果報告書

クリエータ : 伊組烈火

担当 PM : 曾川景介

2025 年 3 月 7 日

目次

1. 要約	1
2. 背景及び目的	2
2.1. 計算機の民主化	2
2.2. Monolithic Kernel とその限界	2
2.3. Microkernel とその限界	3
3. プロジェクト概要	6
4. 開発内容	9
4.1. A9N Microkernel の開発	9
4.1.1. Basic Types	9
4.1.2. API Primitive	10
4.1.3. Capability Overview	11
4.1.4. Capability の操作体系	11
4.1.5. Capability Slot	11
4.1.6. Virtual Message Register	15
4.1.7. Scheduler	15
4.1.8. Kernel-Level Stack	15
4.1.9. Capability Call の略式表記	17
4.1.10. Capability Result	17
4.1.11. Capability Node	19
4.1.12. Generic Capability	27
4.1.13. Address Space Capability	31
4.1.14. Page Table Capability	33
4.1.15. Frame Capability	34
4.1.16. Process Control Block Capability	35
4.1.17. IPC Port Capability	40
4.1.18. Notification Port Capability	50
4.1.19. Interrupt Region Capability	52

4.1.20. Interrupt Port Capability	54
4.1.21. IO Port Capability	56
4.1.22. Virtual CPU Capability	59
4.1.23. Virtual Address Space Capability	63
4.1.24. Virtual Page Table Capability	65
4.1.25. ABI	66
4.1.26. Boot Protocol	68
4.1.27. Init Protocol	72
4.1.28. Build System	74
4.2. Nun Operating System Framework の開発	75
4.2.1. API Library	75
4.2.2. Build System	77
4.2.3. Entry Point	78
4.2.4. Code Exapmle	79
4.3. KOITO の開発	80
4.3.1. KOITO Server	81
4.3.2. User-Level Memory Management Server	82
4.3.3. POSIX Server	83
4.4. liba9n の開発	86
4.4.1. liba9n::std	86
4.4.2. liba9n::option<T>	87
4.4.3. liba9n::result<T, E>	87
4.4.4. Conditionally Trivial Special Member Functions	88
4.4.5. liba9n::not_null<T>	89
4.5. A9NLoader の開発	90
4.5.1. CMake Integration	90
5. 開発成果の特徴	91
5.1. HAL	91

5.2. Capability	91
5.3. 高速な IPC	92
5.4. Nun による OS 開発の簡易化	92
5.5. 先行研究	92
5.5.1. Zircon [1]	92
5.5.2. seL4 [2]	93
5.5.3. A9N の優位点	94
6. 今後の課題, 展望	95
6.1. Document	95
6.2. Porting	95
6.3. POSIX Compatibility	95
6.4. User-Level Virtual Machine Monitor	95
6.5. Future	96
7. 実施計画書内容との相違点	97
8. 成長の自己分析	98
9. 付録	99
9.1. 用語説明	99
9.2. 関連 Web サイト	101
9.3. Sample Codes	102
9.3.1. Nun を用いた Hello World プログラム	102
9.3.2. <code>liba9n::result<T, E></code> を用いた Method Chain	103
参考文献	104

1

要約

本プロジェクトでは、Hardware Abstraction Layer を備えた Capability-Based Microkernel である A9N と、その Kernel 上に Operating System を構築するための Framework である Nun、そして Nun を用いて実装される、ユーザレベルでメモリ管理を実行する Operating System の KOITO を開発した。開発した Microkernel の IPC 性能は世界最速レベルを達成した。

2

背景及び目的

2.1. 計算機の民主化

近年、計算機の民主化による Internet of Things (IoT) の拡大は大きなムーブメントを生み出している。Moore's Law [3] で予見されたように、面積当たりのトランジスタ数は指数関数的に増加した。その結果として、軍需産業や学術研究機関に限られていた計算機の利用は大衆化し、黎明期とは比較にならないほどの賢さを数多の人間が手にすることとなった。

民主化に伴い、その Hardware を司る Kernel や OS といった基盤ソフトウェアの存在感も大きく増している。Kernel は System と Hardware を接続する Interface であり、資源管理の抽象化を担うものである。OS は Kernel の提供する Application Programming Interface (API) を用い、要件達成のための機構を User に提供する。このように、複雑さはこれらの抽象化を行う Layer によって隠蔽される。

2.2. Monolithic Kernel とその限界

しかしながら、既存の Kernel や OS はコードベースの肥大化による問題を抱えている。現在広く使われている Linux Kernel を例とする。2024 年時点で 2000 万行を超えるコードを有する巨大なプロジェクト [4] であり、現在進行系でコードベースは成長を遂げている。現在は OSS コミュニティの“人海戦術”とも言える体制によって保守されているが、このアプローチには問題がある。

まず、人的リソースの限界が挙げられる。計算機技術の進歩により、Kernel や OS の機能は日々拡張されている。現在以上のペースでコードベースが拡大した場合に伴う開発の困難化は明白である。

次に、利害関係者の増加に伴う意思決定スピードの低下も問題である。巨大なプロジェクトは成長とともに関係者も例外なく増加する傾向にあるが、これには開発の停滞化によるスケールアップの困難さが生じる。つい先日、Rust 対応のために Linux Kernel へ DMA API を呼び出す抽象化 Layer を追加する提案が Maintainer によって強く否定される自体が発生した [5]。

また、肥大化による Trusted Computing Base (TCB) の拡大は Security 上のリスクを増大させる。Kernel は Hardware などの抽象度が異なる Layer を直接操作するものであり、最も高い権限を持つ。言い換えれば、Kernel は上に存在するすべての Software から信頼される対象である。特権的なコンポーネントの拡大に伴って攻撃対象は拡大するため、System 全体の信頼性や Security 耐性は低下する。コードの殆どは Device Driver であり、Kernel-Level で多くの外部機構を扱う。したがって、Device Driver が Crash した場合 System 全体がダウンしてしまう。実際、商用 OS である Windows XP が Crash する要因の 85% は Device Driver に起因していた [6]。また、Linux Kernel における Device Driver のバグ率は他の Kernel 部分と比較して 7 倍高いという結果 [7] もある。

このように、一枚岩の Monolithic な Kernel (Figure 1) や OS はもはや現代の要求に適合しない。したがって、これを置き換える新たなアーキテクチャが求められている。

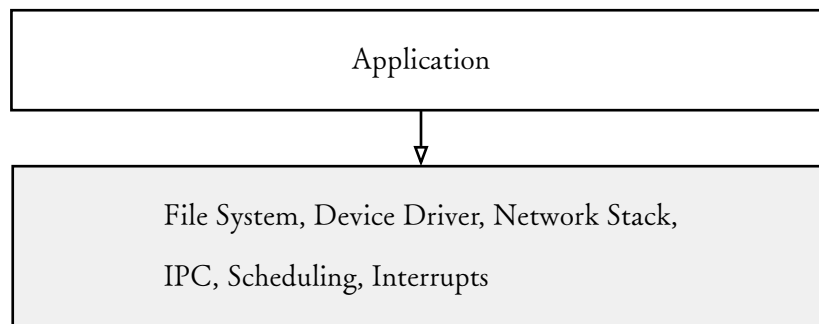


Figure 1: Monolithic Kernel

2.3. Microkernel とその限界

前述の問題を解決するためのアプローチとして有力なのが Microkernel (Figure 2) である。この Kernel は従来の Monolithic な Kernel とは異なり、提供する機構を最小化することで User へ最小限の Primitive のみを提供するような設計手法である。この Architecture では、ほぼ全ての Device Driver や File System, Network Stack といった機能は Kernel Space から User Space へ移動される。したがって、不安定な Device Driver

が Crash しても Kernel がダウンすることはない。また、System の構成を要件に合わせて Dynamic に変更することも容易となる。この仕組みは Policy/Mechanism Separation (PoLP) [8] を満たすものである。

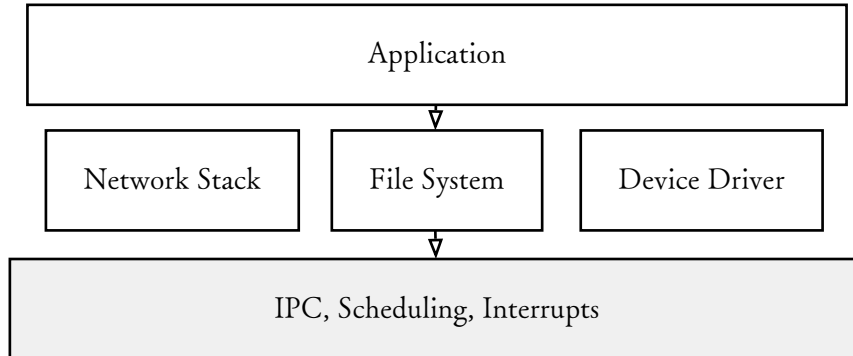


Figure 2: Microkernel

このアプローチは一見完璧なように見えるが、実際にはいくつかの問題が存在する。

一般的に、Microkernel は従来の Monolithic Kernel と比較して性能が劣るとされている。Monolithic Kernel では特権的機能の呼び出しが 1 回の System Call で完結するため非常に高速である。Kernel 内部では Function Call の連鎖に過ぎないためである。しかし、Microkernel では多くの場合 IPC によって処理が複数の Server へ委譲される。IPC は User Space から Kernel Space へ、またその逆の Switch を実行する。したがって、Context Switch の回数が増加し性能が低下する。Single Server と呼ばれる、Device Driver や File System といった機構を単一の Server として実装してしまうことで IPC の実行回数を削減するアプローチも存在するが、不安定さの分離や Dynamic な構成変更といった Microkernel の利点を損なうことになる。

また、Microkernel はすべての機構を User-Level で実行できるわけではなく、Process Scheduling や Memory Management といった機構の分離は困難である。

まず、Clock Interrupt 毎に User-Level Scheduler を呼び出すアプローチには速度面の問題がある。一般的に Clock Interrupt は 100-1000Hz の高頻度で発生するが、IPC の実行は先述した通り高コストである。Overhead を削減するため Linux のように Kernel 内部へ VM を実装する手法 (e.g., eBPF, Wasm) も考えられるが、これでは Microkernel の利点を損なうだけである。

次に、Memory Management ではどうだろうか？ 残念ながら、これには Security の問題がある。Device Driver と同様に Memory Management を User へ移譲することを考

える。Memory の Map や Unmap, Page Table の管理を Kernel-Level ではなく User-Level で実行したい。しかしながら、これでは Kernel に必要な領域を Kernel 自身が確保できなくなってしまう。この問題に対処すべく Kernel に Static な Heap を持たせる手法が考えられるが、使い切ってしまった場合に動作が不可能となる。User-Level の Server に IPC を送信して委譲する手法も考えられるが、これでは委譲先の Server に Kernel の Meta Data が漏洩してしまうため、PoLP [9] を満たすことができない。これらの問題から、多くの Microkernel は Memory Management を Kernel-Level で実装している。

如何にすれば、PoLP を保ったまま柔軟性や安定性を手に入れることができるだろうか？これには新しい手法が必要である！

3

プロジェクト概要

前述したように、既存の Monolithic Kernel-Based System は柔軟に欠け、Security 上のリスクや保守性の低下といった重大な問題を抱えている。また、Microkernel-Based System はこれらの問題を解決しているように見えるが、性能低下や Kernel-Level での Memory Management といった別の問題を抱えている。

本プロジェクトではこれらの問題を解決するために、新たな Security Model と高い性能を兼ね備えた次世代 Microkernel である A9N を開発した。そして、これを Base として OS 構築を支援するための Framework である Nun と、それを用いて実装される OS である KOITO を開発した。これにより、User が Dynamic な Usecase に合わせた Secure な System を構築できる環境を提供する。さらに、A9N Microkernel の開発を継続していくため `liba9n` という Modern な Error Handling 機構を提供する C++20 用の Library を開発した。この Library により、Kernel Developer と Kernel の User という 2 つの立場における、開発の継続性や安全性の妥当性を保証することが可能となる。

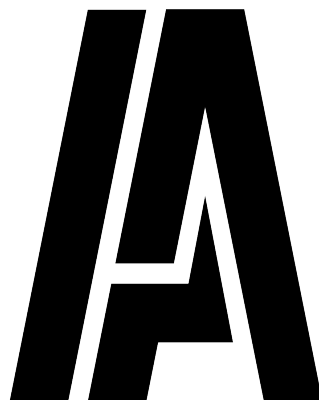


Figure 3: A9N Microkernel のロゴ

A9N Microkernel (Figure 3) は Monolithic Kernel と Microkernel の課題を解決するために設計された, Hardware Abstraction Layer (HAL) による移植容易性を持つ Microkernel である. Object-Capability Model による Capability-Based Security を採用し, 抽象 Memory Capability による Secure な User-Level Memory Management を可能とする. また, 高速な IPC 機構による Low-Latency な通信を実現する. さらに, A9N の Kernel-Level API は Architecture-Independent であるため, Device Driver と Virtual Machine Monitor (VMM) を除くほぼ全ての機構は容易に Porting 可能である.

Nun は A9N Microkernel 上に OS を構築するための Rust 製 Framework である. Kernel は Init Protocol をもって Initial Server を起動するが, この Framework はその Protocol に沿う OS を構築するために最小限の抽象化を提供する. Rust を用いることで Type Safety や Memory Safety を保証し, また Cargo による優れた Build System を享受できる. 通常, Microkernel-Based System の Startup は複雑な処理 Flow を持つが, Nun はこれを簡略化し, User が OS を開発する際の負担を軽減する.



Figure 4: KOITO のロゴ

KOITO (Figure 4) は Nun によって実装される, A9N Microkernel 上で動作する OS である. A9N によって提供される Mechanism を用い, それに対応する Policy を実装する Layer といえる. 実態としては複数の Server で実現される抽象概念であり, Initial Server, POSIX Compatible Server, User-Level Memory Management Server によって構成される. KOITO を構成する各 Server はすべて User-Level で動作するため, 極端な Usecase から普遍的な Usecase まで動的かつ柔軟に対応することが可能となる.

KOITO は実用 OS としての役割と、A9N-Based な OS の Reference 実装としての役割を兼ね備える。したがって、User は新規 OS の開発に際して、KOITO の各 Component を再利用することができる。

liba9n は A9N Microkernel のために設計された C++20 Template Library である。Standard Library に一切依存しないため、Embedded な Freestanding 環境で Modern な Error Handling 機構を使用できる。

- `liba9n::std` は `libc++` の Subset であり、基本機構や Meta Programming のための Utility を提供する。
- `liba9n::option<T>` は Haskell の `Maybe` に相当する型を提供する。Monadic Operation を Support し、逐次処理の高度な抽象化を用いた開発を可能とする。
- `liba9n::result<T, E>` は Haskell の `Either` に相当する型を提供する。
`liba9n::option<T>` と同様に Monadic Operation を Support する。
- `liba9n::not_null<T>` は `liba9n::option<T>` や `liba9n::result<T, E>` で参照を扱うために設計された型である。型レベルで Pointer が Null でないことを保証する。

A9NLoader は A9N Boot Protocol(x86_64)に準拠した Boot Loader の EDK2 による Reference Implementation である。UEFI 搭載の Computer 上で A9N Microkernel と Init Protocol に準拠した Init Server を Memory 上に Load する。その後、Kernel へ Control を委譲して System 全体の Entry Point とする。

4

開発内容

4.1. A9N Microkernel の開発

4.1.1. Basic Types

A9N Microkernel は C++20 を用いて開発されているが、Kernel 内部で広範に使用するための基本型を定義している。Kernel 内部では幅が固定された型を基本的に使用せずに `word` 型を使用する。`word` は Architecture-Specific な Word 幅を持つ符号なし整数型であり、`uintmax_t` や `usize` の Alias として定義される。これにより、速度と移植容易性を実現する。A9N における Kernel の呼び出し機構は C ABI に依存しない Virtual Message Register-Based なものである。したがって、Kernel は多値の返却や正常値とエラー値の区別が可能な形式で User に制御を返すことができる。そのため、言語の Library レベルで Mapper を作成することにより、Native な Result 型やその他の型を返すことができる。このような API の Rust による Reference Implementation は Nun OS Framework に内包されている。

具体的な型定義は以下の通り (cf., Table 1) である。これらは `::a9n` という Namespace に定義される。

name	description
word	Architecture-Specific な幅を持つ符号なし整数型
sword	Architecture-Specific な幅を持つ符号付き整数型
physical_address	Physical Address を表現する型
virtual_address	Virtual Address を表現する型
capability_descriptor	Capability Descriptor (cf., Figure 5)を表現する型

Table 1: Types

また、以下の定数群（cf., Table 2）も同様に定義される.

name	description
BYTE_SIZE	Byte の Size
WORD_SIZE	word の Size
PAGE_SIZE	Frame の Size
BYTE_BITS	1byte の Bit 数
WORD_BITS	word の Bit 数 ; $\text{WORD_SIZE} * \text{BYTE_BITS}$ として定義される

4.1.2. API Primitive

A9N Microkernel は User に対して Kernel Call を提供する. Kernel Call は細分化することができ, 以下 2 + 1 個の API を提供する. これらは従来型 System における System Call に相当するものである:

1. Capability Call
2. Yield Call
3. Debug Call

従来型の System, 例えば Linux Kernel の System Call 数は 2024 年時点で 300 を超える [10] が, A9N はその 1/100 程度で System を構築することが可能である.

4.1.3. Capability Overview

A9N Microkernel の実装には Object-Capability Model [11] による Capability-Based Security を採用し、従来のシステムが抱えていた課題を解消した。Capability は特権的リソース : Object に対するアクセス権限を示す Unique な Token である。従来の ACL を用いたアクセス —— リソース自身が Permission を確認する方式とは異なり、該当 Capability の所有者のみが操作を実行可能である。このように、PoLP を満たしつつも柔軟なアクセス制御を実現する。

言い換えると Capability は Token であり、間接的に Object へアクセスするための Handle である。要するに、あらゆる特権的操作は Object が持つ固有機能の呼び出しとして Model 化される。したがって、Object-Oriented Programming における Object の Method Call と同等に捉えることができる。また、Capability と Object を同一視することもできる。よって、この文書では Capability と Object を同義として扱う。

Capability は複数の Context 間で Copy や Move が可能である。この仕組みにより、User は Capability を Server 間で委譲して特権的な操作の実行範囲を最小化できる。

4.1.4. Capability の操作体系

A9N Microkernel において、操作対象の Capability を指定するために Capability Descriptor と呼ばれる符号なし整数型を用いる。Capability Descriptor は後述する Capability Node を再帰的に探索するための Address である。Capability Call の実行時、First Argument として Capability Descriptor を指定する (Listing 1) ことで Root Capability Node から対象が暗黙的に探索される。

```
fn capability_call(target: capability_descriptor, args ...) ->
    capability_result
```

Listing 1: Capability Call の Pseudo Code

4.1.5. Capability Slot

Capability は内部的に Capability Slot と呼ばれるデータ構造に格納される。Capability Slot は Capability Component への Pointer と Slot Local Data, Capability Rights, Dependency Node から構成される。また、Capability Slot は $\text{WordWidth} * 8$

byte に Align される (e.g., 32bit => 32byte, 64bit => 64byte). したがって, Cache Line 上に乗せやすくなる.

4.1.5.1. Capability Component

すべての Capability を C++ 上で統一的に扱うため, Capability Component という Interface Class を定義する (Listing 2). Capability Component は GoF [12] における Command Pattern と Composite Pattern を統合したものであり, Capability の実行と初期化, 探索を統一的な Interface によって提供する.

```
class capability_component
{
public:
    // command
    virtual capability_result execute(
        process &owner,
        capability_slot &self
    ) = 0;
    virtual capability_result revoke(capability_slot &self) = 0;

    // composite
    virtual capability_lookup_result retrieve_slot(a9n::word index) = 0;
    virtual capability_lookup_result traverse_slot(
        a9n::capability_descriptor descriptor,
        a9n::word descriptor_max_bits,
        a9n::word descriptor_used_bits
    ) = 0;
};
```

Listing 2: Capability Component の Interface

すべての Capability は Capability Component の実装である.

4.1.5.2. Slot Local Data

Slot に Capability Component への Pointer を格納するだけでは問題が生じる. 例えば Process Control Block のような Capability を考えると, これは Component としての Instance ごとに状態を持つため問題は発生しない. しかしながら Memory に関連する

Capability(e.g., Generic, Page Table, Frame)を考えたとき、これらのために1つずつ Unique な Instance を生成しては効率が悪い。よって、そのような Usecase に対応するため Slot Local Data を導入した。対象の Capability は Slot Local Data にそれらの情報を保持し、Capability Component として指す Instance は Capability ごとに単一のものを共有するようなアプローチを取ることができる。これにより、Memory の新規 Allocation を必要とせずに Capability を生成可能とした。この Slot Local Data という仕組みは Memory に関連する Capability に限らず有用であり、どのように利用するかは Capability Component の実装によって決定される。

4.1.5.3. Capability Rights

前述した通り、一部の例外を除いて Capability は Copy や Move が可能である。Capability が Copy された場合、Destination と Source は同一の Capability として扱われる。

しかし、これらの Capability に対して別々のアクセス制御を実行したい Usecase が存在する。典型例として、IPC Port Capability を親が子に共有するが、子からはこの Capability を削除できないようにしたい¹場合がある。このようなシナリオに対応するため、Capability Slot 固有の Capability Rights を導入した。Capability Rights は Capability の Copy や Read, Write に対する挙動を制御するための Bit Flag である (Listing 3)。

```
enum object_rights : uint8_t
{
    NONE    = 0,
    READ    = 1 << 0,
    WRITE   = 1 << 1,
    COPY    = 1 << 2,
    MODIFY  = 1 << 3,
    // MOVE is always allowed
    ALL = READ | WRITE | COPY | MODIFY,
};
```

Listing 3: Capability Rights の定義

¹Dependency Node を除いて親や子といった概念は Kernel に存在しない。これは Kernel を使用する OS Layer でみたときの例である。

Capability Rights には、先天的に設定されるものと後天的に設定するものの両方が存在する。原則として、Capability は生成時点にすべての Rights Bit が設定される。ただし、Copy を許可すると同一性が失われてしまうような Capability は Copy が最初から禁止される。

4.1.5.4. Dependency Node

Capability はその依存関係を Dependency Node (Listing 4) によって管理する。Dependency Node は依存関係にある Capability Slot を保持するが、`depth` によって子と兄弟を区別する。

```
struct capability_slot
{
    // 前略
    capability_slot *next_slot;
    capability_slot *preview_slot;
    a9n::word depth;
    // 後略
};
```

Listing 4: Capability Slot の Dependency Node 部

- 親の区別は可能だが、通常使用されないため省略される。
- `next_slot` もしくは `preview_slot` の `depth` が自分自身の `depth` と等しい場合、その Slot は兄弟である。
- `next_slot` の `depth` が自分自身の `depth` よりも大きい場合その Slot は子である。子は必ず `next_slot` 側に設定されるため、`preview_slot` の `depth` は比較しない。

Dependency Node は所有関係を表すものではなく、あくまでも派生と同一性を表すために利用される。

4.1.6. Virtual Message Register

A9N Microkernel では Capability Call のために Virtual Message Register²機構を使用する。Virtual Message Register はその名の通り、Communication に使用するための Message を格納する Register である。

- Architecture ごとに Virtual Message Register は Hardware Register へ Map される³。
- Hardware Register に Map できない Message⁴は Process Control Block ごとの IPC Buffer に格納される。IPC Buffer は Kernel と User 間の Shared Memory であり、必ず存在が保証される。

このアプローチは高速かつ Secure な Capability Call を実現する。

- Hardware Register への Access は一般に高速であるため、Message Copy の Overhead を最小限に抑えることができる。
- IPC Buffer は Capability によって存在が保証されるため、Kernel Space における User Space 起因の Page Fault は発生しない。

4.1.7. Scheduler

A9N Microkernel は Benno Scheduler [13] を Process Scheduling に使用する。Priority-Based Round-Robin Scheduler であり、255 段階の Priority Level を持つ。基本的には従来の Scheduler と同じだが、Benno Scheduler が異なる点は必ず実行可能な Process のみを Queue に保持するところにある。このアプローチは Queue 操作を簡易化し、なおかつ Hot-Cache 内の実行による高速化を実現することができる。その結果、System は Low Latency となる。

4.1.8. Kernel-Level Stack

A9N Microkernel は Event Kernel Architecture であり、Kernel Stack を CPU コアごとに割り当てる Single Kernel Stack [14] アプローチを採用している。従来の Process Kernel Architecture では実行可能な Context ごとに 4-8KiB の Kernel Stack を割り当てていたが、この方式では大量の Kernel Memory を消費してしまう欠点がある。CPU コアごと

²L4 Microkernel Family における UTCB と同等

³ABI 項を参照

⁴ABI 項を参照

のKernel Stack は Memory Footprint を削減し，実行可能 Context 数の Scalability を向上させる．

4.1.9. Capability Call の略式表記

本文書では各 Capability ごとの Capability Call を略式表記する。通常、Capability Call 全てに共通な引数は以下ようになる：

type	name	description
message_register[0]	target_descriptor	対象 Capability への Descriptor
message_register[1]	operation	対象 Capability に対する操作

また、返り値は以下ようになる：

type	name	description
message_register[0]	is_success	操作の成否
message_register[1]	error	Capability Call の Error ⁵

このように Message Register の Index : 0 と Index : 1 は予約されているが、統合され Capability Result 型 (cf., Listing 5) として使用される。それ以外の Message Register はそれぞれの Capability が定義したように使用できる。

したがって、各 Capability Call の略式表記は以下ようになる：

- 返り値が Capability Result 型の場合、返り値の表記は省略する。返り値が存在する場合はそれを記述するが、Capability Result は省略する。
- operation はそれぞれの Capability Call によって異なるため、その指定を Library Function に内包させる。そのため、表記からは省略する。

4.1.10. Capability Result

Capability Result は `liba9n::result` を用いて定義される Capability Call の返り値であり、以下のように (cf., Listing 5) 定義される。これらは `a9n::kernel` Namespace に内包される。

⁵現在は簡易化のために Capability Error 型 (cf., Section 4.1.10) のみを返しているが、将来的にこの Field も Capability-Defined な値の返却に使用する予定である。

```
enum class capability_error : a9n::word
{
    ILLEGAL_OPERATION,
    PERMISSION_DENIED,
    INVALID_DESCRIPTOR,
    INVALID_DEPTH,
    INVALID_ARGUMENT,
    FATAL,
    DEBUG_UNIMPLEMENTED,
};

using capability_result = liba9n::result<void, capability_error>;
```

Listing 5: Capability Result の定義

4.1.11. Capability Node

Capability Node は Capability を格納するための Capability であり, seL4 Microkernel における CNode の設計をベースとしている. 1 つの Node は $2^{\text{RadixBits}}$ 個の Capability Slot を持ち, この数だけ Capability を格納できる. したがって, 論理的には Capability Node を Capability Slot の配列としてみなすことができる.

Capability Node は効率のために Radix Page Table をベースとした木構造を取る. 仮に単純な Linked List として Capability Node を実装した場合, Capability Slot の探索には $O(n)$ のコストが発生する. 一方, Radix Page Table をベースとした実装を採用することで, Capability Slot の探索を $O(\log n)$ で実現することができる.

Capability Component は `retrieve_slot` と `traverse_slot` を定義するが, この具象となる実装を呼び出すことで Capability Node を探索し, 対象の Capability Slot を取得することができる.

4.1.11.1. `capability_node::retrieve_slot`

このメソッドは `capability_component::retrieve_slot` によって定義される Interface の実装である. `retrieve_slot` は引数に指定された Index に対応する Slot を返す. これは単なる配列アクセスに等しい.

4.1.11.2. `capability_node::traverse_slot`

このメソッドは `capability_component::traverse_slot` によって定義される Interface の実装である. `traverse_slot` Node 間の再帰的な探索であり, 以下のように実装される:

1. Capability Descirptor から Descriptor Used Bits 分を Skip した箇所から Node の Radix Bits 分を取り出し (Listing 6), Node Index とする.
2. Node Index を用いて Slot を取得し, 次の探索対象とする.
3. 3 で取得した Slot から Capability Component を取得し, 再帰的に `taverse_slot` を呼び出す.

```

inline const a9n::word capability_node::calculate_capability_index(
    a9n::capability_descriptor descriptor,
    a9n::word descriptor_used_bits
)
{
    // index用のmask baseを計算する.
    auto mask_bits = static_cast<a9n::word>((1 << radix_bits) - 1);
    // descriptorからradix bitsを取り出すためのshift bitsを計算する.
    // このshift bitsはdescriptorから未使用bitを取り出すために使用する;
    // 要するに使用済みbitをskipする.
    auto shift_bits = (a9n::WORD_BITS -
        (ignore_bits + radix_bits + descriptor_used_bits)
    );
    // 未使用bitの先頭からradix bitsを取り出しindexとする.
    return (descriptor >> shift_bits) & mask_bits;
}

```

Listing 6: Node Index の取得

Node 以外の Capability Component 実装は、`retrieve_slot` や `traverse_slot` の呼び出し時に `capability_lookup_error::TERMINAL` を返す。この機構により、どの Capability Component を呼び出すかに関わらず Capability Node の探索を行うことができる。

4.1.11.3. Addressing

Capability Call の実行時、対象となる Capability は指定された Capability Descriptor を用いて暗黙のうちに Root Capability Node から探索される。User が指定した Capability Descriptor の先頭 8bit は Depth Bits であり (Figure 5)、Capability Node の探索上限を示す。ここでは簡略化のために 64bit Architecture における Descriptor を例示しているが、異なる Word 幅の Architecture においても同様の構造をとる。

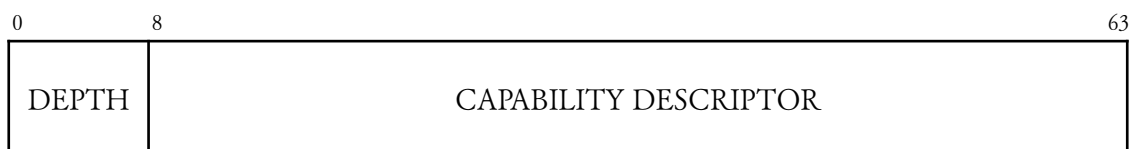


Figure 5: User が指定する Capability Descriptor の構造

Addressing 機構は先述したように Radix Page Table をベースとしているが、具体例を示すことで理解の助けとする。

まず、Node₀(Root Node), Node₁, Node₂ の Capability Node が存在するとする。

Node₀ の Slot 数を 256(*i.e.*, 0xff)個とした場合、Node₀ の Radix Bits は

$$\log_2(256) = 8 \quad (1)$$

となる。

続いて、Node₁ の Slot 数を 1024(*i.e.*, 0x400)個とした場合、Node₁ の Radix Bits は

$$\log_2(1024) = 10 \quad (2)$$

となる。

同様に、Node₂ の Slot 数を 64(*i.e.*, 0x40)個とした場合、Node₂ の Radix Bits は

$$\log_2(64) = 6 \quad (3)$$

となる。

そして、Node₀ の Index : 0x02 に Node₁ を格納し、Node₁ の Index : 0x03 に Node₂ を格納する。また、Node₂ の Index : 0x04 に Node ではない終端の Capability として Capability_{Target} を格納する。

これを図示すると (Figure 6) になる。

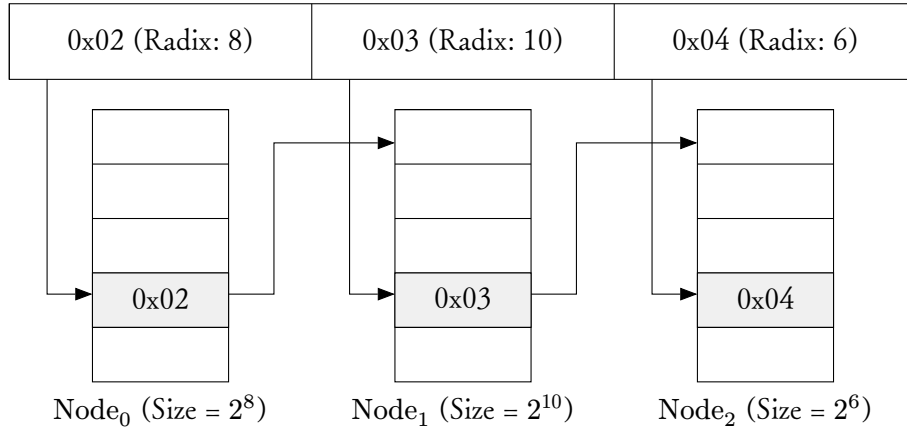


Figure 6: Capability 構成の例

ここで、Capability_{Target} を対象として Capability Call を実行したい場合を考えると、Capability Descriptor は (Equation 4) のようになる。ここでは簡略化のために 32bit Architecture における Descriptor を例示しているが、異なる Word 幅の Architecture においても同様の構造をとる。

$$\begin{aligned} \text{capability_descriptor} &:= 0x180300C5 && (\text{hex}) \\ &:= 0b00011000000000110000000011000101 && (\text{bin}) \end{aligned} \quad (4)$$

これを Node の Radix Bits によってパースすると, (Equation 5) となる:

$$\begin{array}{ccccccc}
 & 8\text{bit} & & 8\text{bit} & & 10\text{bit} & & 6\text{bit} \\
 & \overline{\hspace{1cm}} & & \overline{\hspace{1cm}} & & \overline{\hspace{1.5cm}} & & \overline{\hspace{1cm}} \\
 0b00011000000000110000000011000101 & & & & & & & \\
 & \text{Depth} & & \text{Index}_{\text{Node}_0} & & \text{Index}_{\text{Node}_1} & & \text{Index}_{\text{Node}_2}
 \end{array} \quad (5)$$

まず, 先頭 8bit から Depth Bits が取り出される. この場合は $0b00011000 = 0x24$ となる. Depth Bits の妥当性を示すため, 実際に計算を行う.

$\text{Capability}_{\text{Target}}$ に対応する Depth Bits は (Equation 6) のように計算される:

$$\begin{aligned}
 \text{Depth}(\text{Capability}_{\text{Target}}) &= \text{Radix}(\text{Node}_0) + \text{Radix}(\text{Node}_1) + \text{Radix}(\text{Node}_2) \\
 &= 8 + 10 + 6 = 24
 \end{aligned} \quad (6)$$

ただし, $\text{Capability}_{\text{Target}}$ のように終端まで探索を行う場合, Depth Bits はその最大値を用いることができる (Equation 7):

$$\text{Depth}_{\text{Max}} = \text{WordWidth} - 8 \quad (7)$$

続いて, Node_0 を探索するための Index を取得する. Node_0 の Radix Bits より 8bit を取り出し, 取得した $0x02$ を $\text{Index}_{\text{Node}_0}$ とする. これを用いて Node_0 から Node_1 を得る.

次に, Node_1 を探索するための Index を取得する. Node_1 の Radix Bits より 8bit を取り出し, 取得した $0x03$ を $\text{Index}_{\text{Node}_1}$ とする. これも同様に Node_1 の Index とし, Node_2 を得る.

最後に, Node_2 を探索するための Index を取得する. Node_2 の Radix Bits より 8bit を取り出し, 取得した $0x04$ を $\text{Index}_{\text{Node}_2}$ とする. これにより, 最終的な $\text{Capability}_{\text{Target}}$ が取得される.

次の例として, Node_1 を対象に Capability Call を実行したい場合を考えると, Capability Descriptor は (Equation 8) のようになる:

$$\begin{aligned}
 \text{capability_descriptor} &:= 0x803xxxx \quad (\text{hex}) \\
 &:= 0b0000100000000011 \text{xxxxxxxxxxxxxxxx} \quad (\text{bin})
 \end{aligned} \quad (8)$$

これを Node の Radix Bits によってパースすると, (Equation 9) となる:

$$\begin{array}{ccccccc}
 & 8\text{bit} & & 8\text{bit} & & \text{Remain Bits} \\
 & \overline{\hspace{1cm}} & & \overline{\hspace{1cm}} & & \overline{\hspace{2cm}} \\
 0b0000100000000011XXXXXXXXXXXXXXXX & & & & & & \\
 & \text{Depth} & & \text{Index}_{\text{Node}_0} & & \text{Unused}
 \end{array} \quad (9)$$

これも同様に Depth Bits の妥当性を検証する. この場合, Depth Bits は (Equation 10) のように計算される:

$$\begin{aligned}
 \text{Depth}(\text{Capability}_{\text{Target}}) &= \text{Radix}(\text{Node}_0) \\
 &= 8
 \end{aligned} \quad (10)$$

Depth Bits は Node のような非終端の Capability を指定するために使用される。常に最大値を使用した場合、必ず終端まで探索されてしまうためである。Capability_{Target} の探索と途中までは同様であるが、パース済みの Descriptor が Depth Bits 以上になった時点で探索を終了する。

4.1.11.4. Capability Call

copy

Capability の Copy を実行する。Rights はそのまま Copy される。

type	name	description
capability_descriptor	node_descriptor	対象 Capability Node への Descriptor
word	destination_index	Destination となる Capability を格納して いる Node 内 Index
capability_descriptor	source_descriptor	Source となる Node の Descriptor
word	source_index	Source となる Node の Capability を格納して いる Index

Table 3: copy の引数

move

Capability の Move を実行する。Rights はそのまま Move される。

type	name	description
capability_descriptor	node_descriptor	対象 Capability Node への Descriptor
word	destination_index	Destination となる Capability を格納して いる Node 内 Index
capability_descriptor	source_descriptor	Source となる Node の Descriptor
word	source_index	Source となる Node の Capability を格納して いる Index

Table 4: move の引数

mint

Capability の Mint を実行する. 新しい Rights は元となる Rights の Subset である必要がある.

type	name	description
capability_descriptor	node_descriptor	対象 Capability Node への Descriptor
word	destination_index	Destination となる Capability を格納して いる Node 内 Index
capability_descriptor	source_descriptor	Source となる Node の Descriptor
word	source_index	Source となる Node の Capability を格納して いる Index
capability_rights	new_rights	新しい Rights (cf., @a9n::capability_rights)

Table 5: mint の引数

demote

Capability Rights を不可逆的に降格する。新しい Rights は元となる Rights の Subset である必要がある。

type	name	description
capability_descriptor	node_descriptor	対象 Capability Node へ の Descriptor
word	target_index	対象の Capability を格納 している Node 内 Index
capability_rights	new_rights	新しい Rights (cf., @a9n::capability_rights)

Table 6: demote の引数

remove

Capability を Slot から削除する。Dependency Node に兄弟が存在しない場合、Revoke を実行してから削除する。

type	name	description
capability_descriptor	node_descriptor	対象 Capability Node への Descriptor
word	target_index	削除対象の Capability を格納している Node 内 Index

Table 7: remove の引数

revoke

Capability を初期化/無効化する。

type	name	description
capability_descriptor	node_descriptor	対象 Capability Node への Descriptor
word	target_index	削除対象の Capability を格納している Node 内 Index

Table 8: revoke の引数

4.1.12. Generic Capability

Generic Capability は物理的な Memory を抽象化した Capability である。Generic は Base Address, Size Radix Bits, Watermark, そして Device Bits から構成される。

- Base Address は Generic が指す Memory Region の開始 Physical Address である。
- Size Radix Bits は Generic が指す Memory Region の Size を示す Radix であり, $2^{\text{SizeRadixBits}}$ が実際の Size である。この事実から分かるように, Generic の Size は必ず 2 の累乗 byte である。
- Watermark は Generic の使用状況を示す Physical Address である。
- Device Bits は Memory Region が Device のために使用されるような場合(e.g., MMIO) に設定される。

Generic Capability はすべての Capability を生成するための Factory として機能する。Convert 操作によって Generic Capability の領域を消費し, 新たな Capability を生成することができる。生成した Capability は Dependency Node へ子として設定され, 破棄の再帰的な実行に利用される。

4.1.12.1. $\log_2$ Based Allocation

Generic の Convert 操作時, 次のステップで Capability を生成する:

1. Convert 操作によって指定された Capability Type と Specific Bits から Size Radix を得る。
2. Size Radix 分を Allocate 可能か確認する。
3. Watermark を Size Radix に Align(Ceil) し, Watermark に Size Radix 分を加算する。

まず, 引数として与えられた Capability Type と Specific Bits から Size Radix を取得する。Capability Object の Size を最も近い 2 の累乗に切り上げ, 2 を底とする対数をとる (Figure 7)。

$$\text{SizeRadix} = \lceil \log_2(\text{Sizeof}(\text{Object})) \rceil \quad (11)$$

Figure 7: Size Radix の計算

Specific Bits が必要となる理由は, Specific Bits によって全体としての Size が決定される Capability Node のような Capability が存在するためである。

次に、Size Radix 分の Memory 領域が Allocate 可能か確認する。Allocate した場合の Watermark を計算し (Figure 8),

$$\text{NewWatermark} = \text{SizeRadix} \cdot \left\lceil \frac{\text{Watermark}}{\text{SizeRadix}} \right\rceil \quad (12)$$

Figure 8: Size Radix に Align された Watermark を計算

それが範囲内か確認する (Figure 9).

$$\begin{aligned} \text{NewWatermark} &< \text{Watermark} + 2^{\text{RadixBits}} \wedge \\ \text{NewWatermark} + 2^{\text{SizeRadix}} * \text{Count} &\leq \text{BaseAddress} + 2^{\text{RadixBits}} \end{aligned} \quad (13)$$

Figure 9: Allocation のための境界チェック

そして、最後に Allocate を実行する。

このように、すべての Capability は Allocate 時に Align される。そのため、Generic を適切に分割してから Capability を Convert することで自然と SLAB Allocator のような振る舞いを実現する⁶。

4.1.12.2. Deallocation

Generic の再利用には、Convert されたすべての Capability を Remove する必要がある。これは Generic に対して Revoke を実行することで再帰的に行われる。すなわち、ある Capability_A を Convert したあとに Capability_B を Convert し、Capability_A を Remove しても Capability_A が使用していた領域を再利用できない。これは Generic の構造を考えれば明らかである。Generic は単純化と高速化のために単一の Watermark のみで使用量管理を実現している。したがって、高粒度な再利用を Kernel は提供しない。その実現には、Generic から再利用単位ごとに子となるような Generic を生成する必要がある⁷。

4.1.12.3. Capability Call

convert

Generic Capability の領域を指定された Capability Type に変換する。

⁶ある Capability の Convert によって Align が発生すると、次の同一 Capability を Convert する際に隙間なく Allocate できるため。

⁷この実装は完全に User-Level で実現される。

type	name	description
capability_descriptor	generic_descriptor	対象 Generic への Descriptor
capability_type	type	生成する Capability の Type
word	specific_bits	Capability 生成時に使 用する固有 Bits cf., Table 10
word	count	生成する Capability の 個数
capability_descriptor	node_descriptor	格納先 Node への Descriptor
word	node_index	格納先 Node の Index

Table 9: convert の引数

Specific Bits (Table 10) は Capability Type 依存の初期化に使用する値である。例えば, Capability Node を Convert する時に指定する Specific Bits は Node の Radix となる。

name	description
Capability Node	Node の Slot 数を表す Radix ($\text{count} = 2^{\text{specific_bits}}$)
Generic	Generic の Size を表す Radix ($\text{size} = 2^{\text{specific_bits}}$)
Address Space	-
Page Table	depth
Frame	-
Process Control Block	-
IPC Port	-
Notification Port	-
Interrupt Region	-
Interrupt Port	-
Virtual CPU	-
Virtual Address Space	-
Virtual Page Table	-

Table 10: specific_bits の定義

4.1.13. Address Space Capability

Address Space Capability は Virtual Address Space を抽象化した Capability である。すべての実行可能な Context は Address Space Capability を持ち、Context Switch 時に切り替えることで Address Space を切り替える。異なる 2 つの Process Control Block に同一の Address Space Capability を設定することで、同一の Virtual Address Space を共有し、いわゆる Thread を User-Level で実現することができる。

Address Space Capability には Page Table Capability や Frame Capability を Mapping 可能である。これにより、User-Level で Virtual Memory Management を実現することができる。

4.1.13.1. Capability Call

map

Page Table や Frame を Virtual Address Space に Map する。

type	name	description
capability_descriptor	target_descriptor	対象 Address Space への Descriptor
capability_descriptor	memory_descriptor	対象に Map する Page Table もしくは Frame への Descriptor
virtual_address	address	Map する Virtual Address
memory_attribute	attribute	Map に使用する属性

Table 11: map の引数

unmap

Page Table や Frame を Virtual Address Space から Unmap する。

type	name	description
capability_descriptor	target_descriptor	対象 Address Space への Descriptor
capability_descriptor	memory_descriptor	対象から Unmap する Page Table もしくは Frame への Descriptor
virtual_address	address	Unmap する Virtual Address

Table 12: unmap の引数

get_unset_depth

Address Space に Virtual Address を Map するうえで、まだ Map されていない Page Table の Depth を取得する。

type	name	description
capability_descriptor	target_descriptor	対象 Address Space への Descriptor

Table 13: get_unset_depth の引数

type	name	description
word	depth	Map されていない Page Table の Depth

Table 14: get_unset_depth の返回值

4.1.14. Page Table Capability

Page Table Capability は Page Table をそのまま抽象化した Capability である。Page Table Capability は Address Space Capability に Map 可能であり、Virtual Address Space に対する Page Table の Mapping を行う。使用時に Architecture-Specific な知識を必要とせず、階層構造は Depth によって管理される。

x86_64 における Page Table を例示する。x86_64 Architecture は通常 4 レベルの Page Table を持つ。まだ Page Table が Map されていない状態を仮定して Virtual Address を Map することを考える。

1. PML4 は Address Space Capability そのものである。
2. PDPT は Depth : 3 の Page Table Capability である。
3. PD は Depth : 2 の Page Table Capability である。
4. PT は Depth : 1 の Page Table Capability である。

以上 3 つの Page Table を Address Space に Map 後、Address Space の `get_unset_depth` を実行すると 0 が返される。Depth : 0 は Frame Capability に対応するため、これを Map することで Virtual Address Space に対する Mapping が完了する。

4.1.14.1. Architecture-Independent な Virtual Memory Management

Architecture-Specific な知識を必要としない Portable な Virtual Memory Management Server を実現する場合、典型的にはまず空の Address Space Capability に対して `get_unset_depth` を実行することが推奨される。ここで得た値はそのまま必要な Page Table の数と Depth に対応するためである。

もちろん、簡易化のために初めから Depth を指定して Page Table Capability を生成することも可能である。このような実装は System の Portability を損なうが、Project 開始時の Prototype としては有用である。

4.1.14.2. Capability Call

現時点で Page Table Capability に Capability Call は存在しないが、将来的にそれ自体の Depth を確認するための `get_depth` が追加される予定である。

4.1.15. Frame Capability

Frame Capability は Physical Memory Frame (i.e., Page)を抽象化した Capability である。Frame Capability も Page Table Capability と同様に Address Space Capability に Map 可能である。したがって、同じ Frame を複数の Address Space に Map することで Shared Memory を実現することができる。

4.1.15.1. Capability Call

get_address

対象 Frame が指す Physical Memory Region の先頭 Physical Address を得る。

type	name	description
capability_descriptor	memory_descriptor	対象 Frame への Descriptor

Table 15: get_address の引数

type	name	description
physical_address	address	Frame が指している Physical Address

Table 16: get_address の返り値

4.1.16. Process Control Block Capability

Process Control Block は、従来の System における Process を抽象化した Capability である。Hardware Context と Time Slice、そしていくつかの Capability を持ち、Scheduler によって Schedule される対象である。ただし、従来の概念とは異なり提供する機構が最小に保たれる。したがって、Process や Thread といった概念の実現には User-Level での適切な Configuration が必要である。

4.1.16.1. Capability

Process Control Block にはいくつかの Capability を Configuration することができる：

Root Node

Process Control Block が使用する Root となる Capability Node。この Process Control Block が Capability Call を実行したとき、指定された Capability Descriptor は Root Node を起点に探索される。

Root Address Space

Process Control Block の Virtual Address Space が規定される Address Space Capability。この Capability を起点として Address Space の Switch が行われ、また Virtual Memory Management が実現される。

Buffer Frame

IPC Buffer として使用する Frame Capability。Frame Capability を用いることで Buffer の存在を保証できる。したがって、安全に Kernel-User 間の Communication を実現できる。

Resolver Port

Process Control Block の実行中に Exception が発生した場合に、その Status を Fault Call として送信するための IPC Port Capability。Exception Status Message を受信した対象はその内容に応じて適切な処理を行い、Exception の発生元を再開できる。Exception の発生時に Resolver Port が設定されていない場合は Double Fault として動作を停止する。

Process Control Block に必要な Capability を Configuration 後⁸, Resume 操作を実行することで Scheduler の Queue に登録される. Benno Scheduler は前述の通り実行可能な Context のみを Queue として持つため, Priority や Time Slice が考慮されたあとに実行が行われる.

4.1.16.2. Capability Call

configure

Process Control Block を Configuration する.

⁸Root Address Space は必須だが, その他の Capability はあくまでも Optional である

type	name	description
capability_descriptor	process_control_block	対象 Process Control Block への Descriptor
configuration_info	info	cf., Figure 10
capability_descriptor	root_page_table	Root Page Table への Descriptor
capability_descriptor	root_node	Root Node への Descriptor
capability_descriptor	frame_ipc_buffer	IPC Buffer として Frame への Descriptor
capability_descriptor	notification_port	Notification Port への Descriptor
capability_descriptor	ipc_port_resolver	Resolver として IPC Port の Descriptor
virtual_address	instruction_pointer	Instruction Pointer
virtual_address	stack_pointer	Stack Pointer
virtual_address	thread_local_base	Thread Local Base
word	priority	優先度
word	affinity	SMP 環境における Affinity (CPU Core の Index)

Table 17: configure の引数

Performance のため, Process Control Block における各 Parameter は Configuration Info (Figure 10)によって一括して Configuration できる.

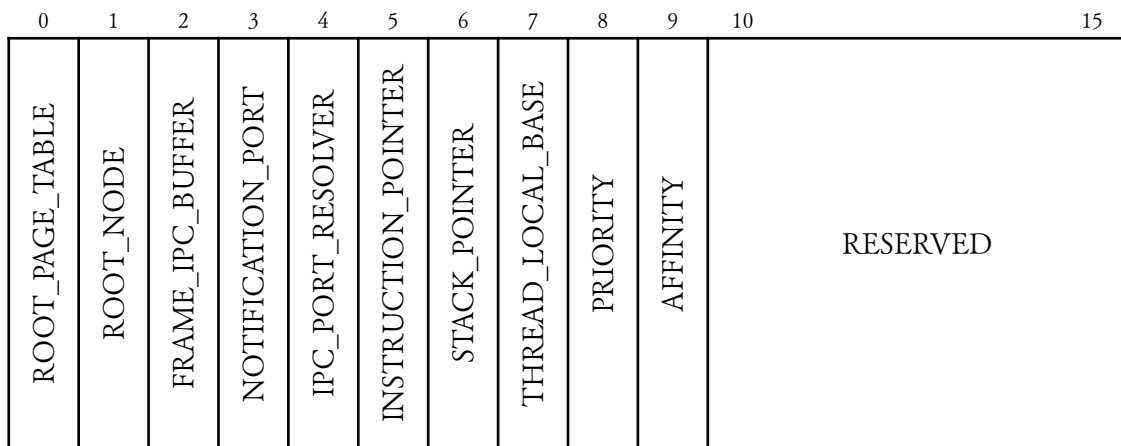


Figure 10: configuration_info の構造

Configuration Info の各 Bit がそれぞれの Field に対応する。この Bit が立っている場合、その Field が Configuration される。逆に言えば、立っていない場合その Field に対応する引数は無視される。

read_register

Process Control Block の Register (Hardware Context) を読み出す。読み出した Register は Message Register の Index:⁹3 から n¹⁰へ格納される。

type	name	description
capability_descriptor	process_control_block	対象 Process Control Block への Descriptor
word	register_count	読み出す Register の数

Table 18: read_register の引数

type	name	description
word[n]	registers	読み出した Register の値

Table 19: read_register の返回值

write_register

⁹Index:0 と Index:1 は Capability Result によって予約されている。また、そのまま write_register を実行してコピーを可能とするために Index:2 も予約されている。

¹⁰Architecture に依存。詳細は ABI を参照。

Process Control Block に Register (Hardware Context) を書き込む。 Message Register の Index:3 から Index:n を読み出し、対象 Process Control Block の Register に書き込む。

type	name	description
capability_descriptor	process_control_block	対象 Process Control Block への Descriptor
word	register_count	書き込む Register の数
word[n]	registers	書き込む Register の値

Table 20: write_register の引数

resume

Process Control Block を実行可能状態にし、Scheduler の Queue に追加する。

type	name	description
capability_descriptor	pcb_descriptor	対象 Process Control Block への Descriptor

Table 21: resume の引数

suspend

Process Control Block を休止状態にする¹¹。

type	name	description
capability_descriptor	pcb_descriptor	対象 Process Control Block への Descriptor

Table 22: suspend の引数

¹¹ 休止状態の Process Control Block は Queue に追加されない。したがって、明示的に再開するまで実行されない。

4.1.17. IPC Port Capability

A9N Microkernel は IPC Port による Rendezvous Indirect IPC を採用している。ある実行可能な Context が IPC Port へ Message を Send すると、同じ IPC Port を持つ他の Context がその Message を Receive できる。Sender と Receiver は 1:n もしくは n:1 の関係を持つ。

例えばある IPCPort_A が存在したとして、 Context_A が IPCPort_A に Send 操作を実行したとする。この状態では Receiver となる Context が存在しないため、 Context_A は Block され、 IPCPort_A の Wait Queue に追加される。さらに Context_B が IPCPort_A へ Send 操作を実行すると、やはり Receiver が存在しないため Block され、 IPCPort_A の Wait Queue に追加される。ここで、Receiver となる Context_C が IPCPort_A へ Receive 操作を実行すると、 IPCPort_A が持つ Wait Queue の先頭から Context_A が取り出され、 Context_A の持っていた Message が Context_C に Copy される。この例を図示すると (Figure 11) のようになる。

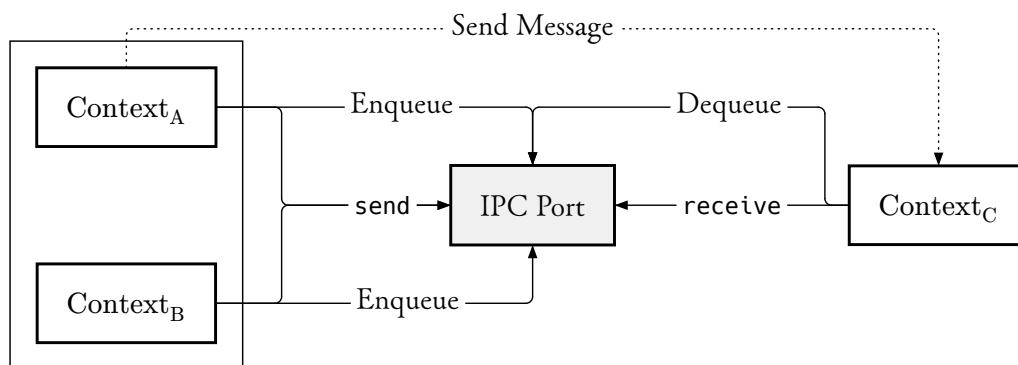


Figure 11: IPC Send/Receive Example

この例は Sender:Receiver が n:1 の場合を示すが、Sender:Receiver が 1:n の場合も同様である。

ここで重要なのが、IPC Port は Message を Buffering しないという事実だ。IPC Port は Sender/Receiver の Queue を保持するが、これは Message の Queue ではない。Message は Sender の Virtual Message Register から Receiver の Virtual Message Register へ直接 Copy されるため高速である。

4.1.17.1. Non-Blocking IPC

Send や Receive 操作は Non-Blocking で実行することも可能である。基本的には前節と同様だが、Queue に Sender/Receiver が存在しない場合 Block せず即座に Return する。

4.1.17.2. Call/Reply Mechanism

先述した Send や Receive 操作は一方向の通信であり、双方向通信にはコストが発生するため基本的に推奨されない。そのため、IPC Port は Call と Reply, Reply Receive という Client-Sever Model (Figure 12) に特化した操作の仕様が推奨される。

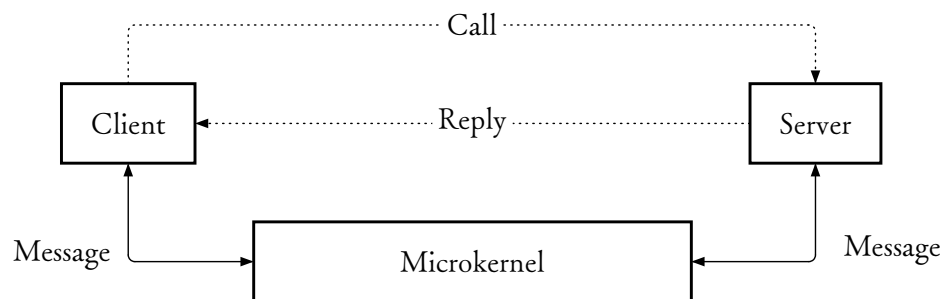


Figure 12: Client-Server Model

これらは双方向通信を 1 つの Path で実行するための操作であり、これによって Context Switch のコストを抑え、なおかつ高度な最適化を実現する。

call

IPC Port から取得した Context に対して Send し Reply を待つ。概念的には Send と Receive を組み合わせたものに近いが、この操作を実行した Context から見たとき Atomic に送受信が実行される点が異なる。言い換えると、Message を Send した対象である Receiver から Message を受信することを保証できる。

この操作を実現するため、Call 時にその Caller は Receiver へ Reply Object を設定する。この Reply Object は Reply State と Reply Target 部によって構成される。

Reply State は Reply Object の状態を示し、Source と Destination の 2 つ存在する。また、それぞれに付随する Context を保持するための Pointer が Reply Target へ格納される。

- Call を実行し、Reply を待っている場合 Source Reply Object に WAIT が設定される。また、送信先の Context を Source Reply Target に設定する。

- Receive を実行し、Reply 先の Context が決定された場合 Destination Reply Object に READY_TO_REPLY が設定される。また、送信元の Context を Destination Reply Target に設定する。

Source Reply Target は一見不要に思えるが、これは無効な参照の発生を避けるために使用される。仮に Call を実行した先の Receiver が途中で破棄された場合、Destination Reply Target が無効な Context を指すことになる。したがって、Destination Reply Object が設定されているような Context を破棄する場合は Source Reply Target を参照し Call を中止する必要がある。

これらの構造を統合したものを(Figure 13)に示す。

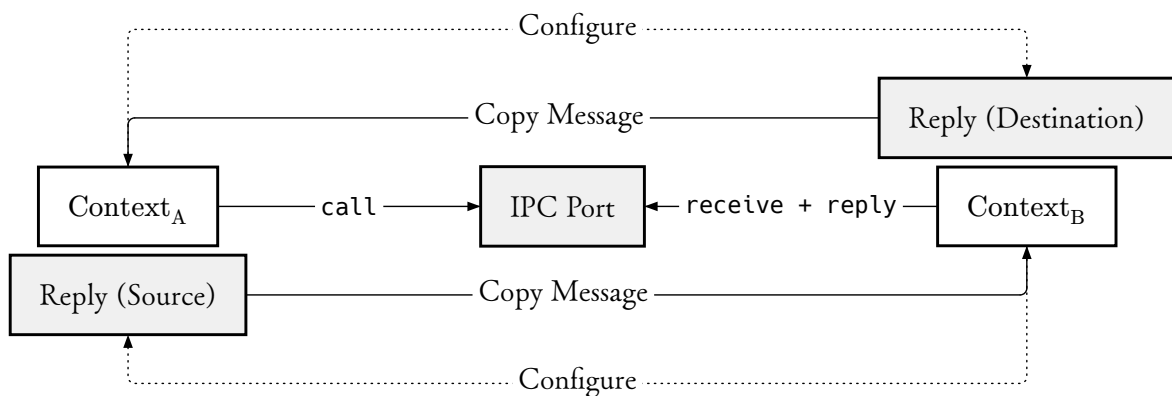


Figure 13: Call/Reply Mechanism

reply

Call に対して Receive を実行した場合、先述した通り Destination Reply Object に READY_TO_REPLY と Caller Context が設定される。Reply はこの Destination Reply Object を参照して Message を送信する。言い換えると、設定されていない場合はすぐさま Return する。これも先述した通り、Receive 先の Context と Reply 先の Context は同一であることが保証される。

Reply Object の存在により IPC Port を介さない、直接的な Message の送信が可能となる。Capability Call の仕様上 IPC Port への Descriptor を指定する必要があるが、この IPC Port は文字通りどの IPC Port を指すものでも良い。

reply_receive

Reply Receive は Reply と Receive を 1 つの Path で実行し、高速に Server を実装するための操作である。

典型的に、Microkernel における Server は以下のような Event-Driven Architecture をとる：

1. Receive を実行し、Reply 先の Context を決定する.
2. 受信した Message を解析し、どのような処理を行うか決定する.
3. Reply を実行し、解析結果ごとの処理結果を Client に返す.
4. 1 に戻る.

疑似コードを図示すると（Listing 7）のようになる.

```
void main()  
{  
    message received_message;  
    message reply_message;  
  
    while (true)  
    {  
        received_message = receive();  
  
        switch(received_message.tag)  
        {  
            // TAG-Specificな処理を行う  
            case TAG_A:  
                reply_message = process_tag_a(received_message);  
                break;  
            case TAG_B:  
                reply_message = process_tag_b(received_message);  
                break;  
            default:  
                return;  
        }  
  
        reply(reply_message);  
    }  
}
```

Listing 7: Microkernel における典型的な Server の Pseudo Code

このうち、Receive と Reply は結合し、Context Switch や Cache Miss のコストを削減できる。ただし、上記の Pseudo Code の順序をそのままに実装することはできない。A9N Microkernel では Virtual Message Register を IPC の送受信ともに共通で使用するためである。そのため、Receive Reply のような操作として実装してしまうと Reply するための Message が Receive によって上書きされてしまう。したがって、二者の順序を入れ替え、Reply を先に実行することでこの問題を回避する。すると、(Listing 8) で示すような Startup を目的とする Receive が必要である。

```
void main()
{
    // あらかじめReceiveを実行しておく
    message received_message = receive();
    message reply_message;

    while (true)
    {
        switch(received_message.tag)
        {
            // TAG-Specificな処理を行う
            case TAG_A:
                reply_message = process_tag_a(received_message);
                break;
            case TAG_B:
                reply_message = process_tag_b(received_message);
                break;
            default:
                return;
        }

        // Loop終盤でReply Receiveを実行する
        reply_receive(reply_message);
    }
}
```

Listing 8: Reply Receive の Pseudo Code

4.1.17.3. Direct Context Switch

Microkernel において IPC は極めて Critical な操作であり、可能な限り Low Latency で実行する必要がある。そのため Direct Context Switch [13] を採用し、可能な限り¹²Sender から Receiver へ、またその逆の Context を直接 Switch する。

4.1.17.4. Identifier

同じ IPC Port を共有(Copy)することで IPC は実現されるが、どの Context から Message が送信されたかを判別するために Identifier 機構を実装した。Identifier は Fiasco.OC における Label や seL4 における Badge に相当する、Kernel によって正当性が保証される Capability Slot 固有の値である。

送信元の識別を如何に実装するかを考える。通常の Microkernel において Sender の識別は PID や TID によって行われるが、A9N Microkernel は Global な ID を採用していない。したがって異なるアプローチが必要である。Kernel-Level の PID や TID が存在しない場合、IPC Message のある領域を OS が予約し ID とする実装が考えられる。しかしこれは単なる Protocol に過ぎず、各 Context は自由に改竄できるため信頼性が低い。これを解決するのが Identifier であり、User-Level における柔軟かつ Secure な Context の識別を実現する。

IPC Port に対する Identify 操作により、Word 型の値を IPC Port(が格納されている Slot)に設定できる。この値は Slot Local Data (Section 4.1.5.2) に格納されるため、同じ IPC Port を共有する Context ごとに設定可能¹³である。そして、この値は IPC の各操作ごとに Kernel の手で転送される。

ある IPC Port の Identifier を書き換え不可にするためには、Capability Node の Mint や Demote 操作によって Capability Rights (Section 4.1.5.3) から Modify Bits を剥奪するだけでよい。この機構により、User-Level で Context が持つ ID の信頼性を保証できる。

4.1.17.5. Capability Transfer

A9N Microkernel は IPC を通じて Capability を転送 (Copy) できる。

¹²Scheduler に対象よりも高い優先度の Context が存在せず、なおかつ実行可能な場合を指す。

¹³実際には Slot Level で設定可能なため、各 Context が Identifier を複数個持つこともできる。

- 送信者は IPC Buffer の Transfer Source Descriptors Field に転送したい Capability Descriptor を設定する。転送の成功時、この Field は 0 にリセットされる。
- 受信者は IPC Buffer の Transfer Destination Node Field に転送された Capability を格納する Node への Capability Descriptor を設定し、また Transfer Destination Index Field に格納先 Node の Index (Offset) を設定する。

Capability Transfer は必ず IPC Buffer を介して行われるため、Virtual Message Register (Section 4.1.6) における Hardware Register が使用されない。したがって、Hardware Register に格納可能な Message のみで完結する IPC よりもやや低速である。

4.1.17.6. Data Structure

message_info

IPC の細かい挙動を制御するための構造である (cf., Figure 14)。送信者はこの構造を設定し、受信者は受け取ることによって情報を取得できる。

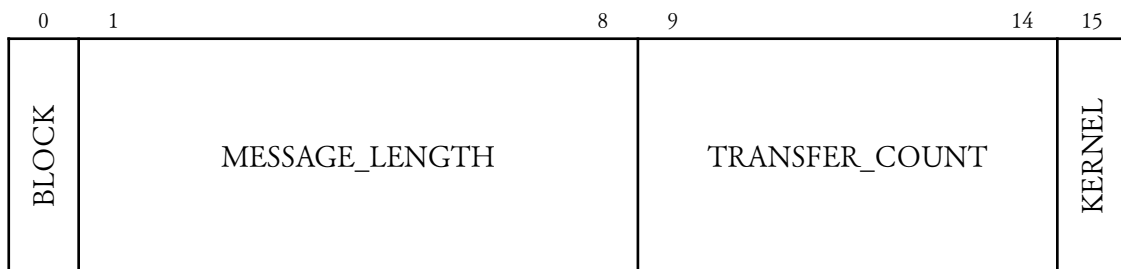


Figure 14: message_info の構造

name	description
BLOCK	設定されている場合、IPC 操作は Block される
MESSAGE_LENGTH	Message の長さ (WORD_BITS 単位)
TRANSFER_COUNT	Transfer する Capability の数
KERNEL	設定されていた場合、Message は Kernel からのものであることを示す。基本的には Fault Call のために使用され、User がこの Flag を設定しても無視される。

4.1.17.7. Capability Call

A9N Microkernel における IPC は第一級の Kernel Call ではなく、あくまでも IPC Port に対する Capability Call として提供される。

send

IPC Port に Message を送信する。

type	name	description
descriptor	ipc_port_descriptor	対象 IPC Port への Descriptor
message_info	info	送信する Message の情報 (cf., Figure 14)

Table 23: send の引数

receive

IPC Port から Message を受信する。

type	name	description
descriptor	ipc_port_descriptor	対象 IPC Port への Descriptor

Table 24: receive の引数

type	name	description
message_info	info	受信した Message の情報 (cf., Figure 14)
word	identifer	送信元の Identifier
word[n]	messages	受信した Message

Table 25: receive の戻り値

call

IPC Port に対して Call を実行する。

type	name	description
descriptor	ipc_port_descriptor	対象 IPC Port への Descriptor
message_info	info	送信する Message の情報 (cf., Figure 14)

Table 26: call の引数

type	name	description
message_info	info	受信した Message の情報 (cf., Figure 14)
word	identifer	送信元の Identifier
word[n]	messages	受信した Message

Table 27: call の戻り値

reply

IPC Port に対して Reply を実行する.

type	name	description
descriptor	ipc_port_descriptor	対象 IPC Port への Descriptor ¹⁴
message_info	info	送信 (Reply) する Message の 情報 (cf., Figure 14)

Table 28: reply の引数

reply_receive

IPC Port に対して Reply Receive を実行する.

¹⁴前述したように, Reply 時に指定する IPC Port はどの IPC Port でも機能する.

type	name	description
descriptor	ipc_port_descriptor	対象 IPC Port への Descriptor
message_info	info	送信 (Reply) する Message の 情報 (cf., Figure 14)

Table 29: reply_receive の引数

type	name	description
message_info	info	受信した Message の情報 (cf., Figure 14)
word	identifer	送信元 (Caller) の Identifier
word[n]	messages	受信した Message

Table 30: reply_receive の戻り値

identify

IPC Port に対して Slot-Local な Identifier を設定する.

type	name	description
descriptor	ipc_port_descriptor	対象 IPC Port への Descriptor
word	identifier	IPC Port に付与する Identifier

Table 31: identify の引数

4.1.18. Notification Port Capability

Notification Port は Asynchronous Notification を実現するための Capability である。Notification Port は IPC Port とは異なり、1Word の Notification Flag Field のみを持つ。

4.1.18.1. Identifier

IPC Port の Identifier (cf., Section 4.1.17.4) と同じ Identifier 機構を持つ。

4.1.18.2. Capability Call

notify

Notification Port に対して Notification を送信する。Slot-Local な Identifier は Notification Flag Field に Bitwise OR される。

type	name	description
descriptor	notification_port_descriptor	対象 Notification Port への Descriptor

Table 32: notify の引数

wait

Notification Port に対して Notification が発生するまで Block する。Notification が発生した場合 Identifier が Bitwise OR された Notification Flag Field が返され、その後 Field は 0 にリセットされる。

type	name	description
descriptor	notification_port_descriptor	対象 Notification Port への Descriptor

Table 33: wait の引数

type	name	description
message_info	info	受信した Message の情報
word	flag	Notification Port の Flag Field

Table 34: wait の戻り値

poll

Notification Port の Notification を Non-Blocking で取得する。Identifier が設定されていない場合の Notification も対象となる¹⁵。

type	name	description
descriptor	notification_port_descriptor	対象 Notification Port への Descriptor

Table 35: poll の引数

type	name	description
message_info	info	受信した Message の情報
word	flag	Notification Port の Flag Field

Table 36: poll の戻り値

identify

Notification Port に対して Slot-Local な Identifier を設定する。

type	name	description
descriptor	notification_port_descriptor	対象 Notification Port への Descriptor
word	identifier	IPC Port に付与する Identifier

Table 37: identify の引数

¹⁵Identifier が設定されていない場合、対象となる Identifier は自動的に 0 として扱われる。この状態で Bitwise OR をしても Flag Field は変化しないが、Notification が発生したという情報はそのまま伝達される。

4.1.19. Interrupt Region Capability

Interrupt Region は割り込みを抽象化する Interrupt Port を生成する (cf., Figure 15) ための Capability である。この Capability は Generic から Convert できず、Init Server の起動時に Init Protocol を通じて 1 つだけ生成される。Generic を介さないため Memory Region を消費することはない。

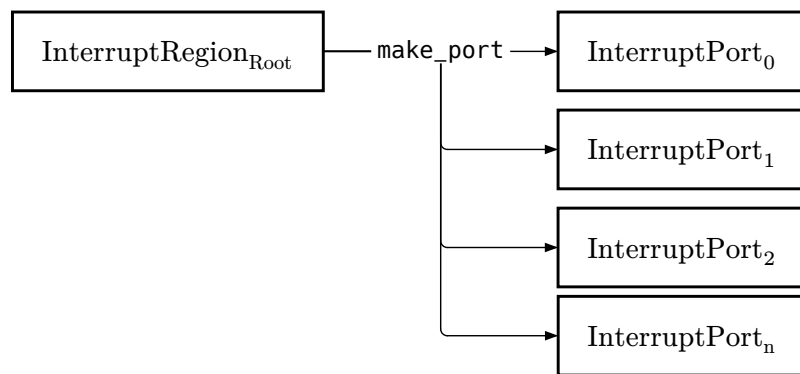


Figure 15: Interrupt Region から Interrupt Port を生成する例

4.1.19.1. Capability Call

make_port

Interrupt Region から Interrupt Port を生成する。この操作によって生成される Interrupt Port は IRQ Number ごとに一意である。

type	name	description
descriptor	interrupt_region_descriptor	対象 Interrupt Region への Descriptor
word	irq_number	生成する Interrupt Port の IRQ Number
capability_descriptor	node_descriptor	Interrupt Port を格納 する Node への Descriptor
word	index	Interrupt Port を格納 する Node の Index (Offset)

Table 38: make_port の引数

4.1.20. Interrupt Port Capability

Interrupt Port は Interrupt Region から生成される Capability である。特定の IRQ Number と 1:1 で対応しており、割り込み発生時に指定した Notification Port へ Notification を送信する (cf., Figure 16).

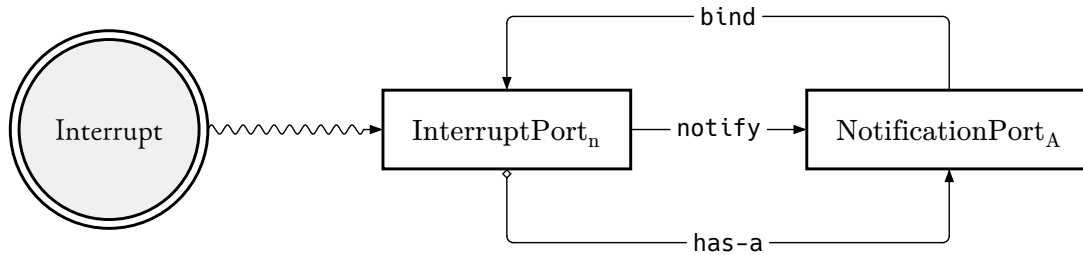


Figure 16: Interrupt Port から Notification Port への Notification を送信する例

4.1.20.1. Capability Call

bind

Interrupt Port に Notification Port を Bind する。割り込み発生時、この Notification Port へ Notification が送信される。

type	name	description
capability_descriptor	interrupt_port	対象 Interrupt Port への Descriptor
capability_descriptor	notification_port	Bind する Notification Port への Descriptor

Table 39: bind の引数

unbind

Interrupt Port に Bind されている Notification Port を Unbind する。

type	name	description
capability_descriptor	interrupt_port	対象 Interrupt Port への Descriptor

Table 40: unbind の引数

ack

Interrupt Port に対して割り込みを Ack する。この操作によって割り込みを再度有効化する。

type	name	description
capability_descriptor	interrupt_port	対象 Interrupt Port への Descriptor

Table 41: ack の引数

get_irq_number

Interrupt Port が対応する IRQ Number を取得する。

type	name	description
capability_descriptor	interrupt_port	対象 Interrupt Port への Descriptor

Table 42: get_irq_number の引数

type	name	description
word	irq_number	Interrupt Port が対応する IRQ Number

Table 43: get_irq_number の戻り値

4.1.21. IO Port Capability

IO Port は Port-Mapped IO (i.e., PMIO) を抽象化する Capability である。PMIO の存在は Architecture に依存する。したがって、PMIO が存在しない Architecture において読み書き操作は無視される。IO Port も Interrupt Port と同様に Generic から Convert できず、Init Server の起動時に Init Protocol を通じて 1 つだけ生成される。また、Generic を介さないため Memory Region を消費しない。

4.1.21.1. Region

IO Port は IO Address Region を持ち、この範囲の Address に対してのみ読み書き操作が可能である。また、IO Port は自身の Subset である IO Port を Mint 操作によって生成することができる (cf., Figure 17)。これにより、特定の Address に対してのみ読み書き操作を許可することができ、特権の最小化を実現する。

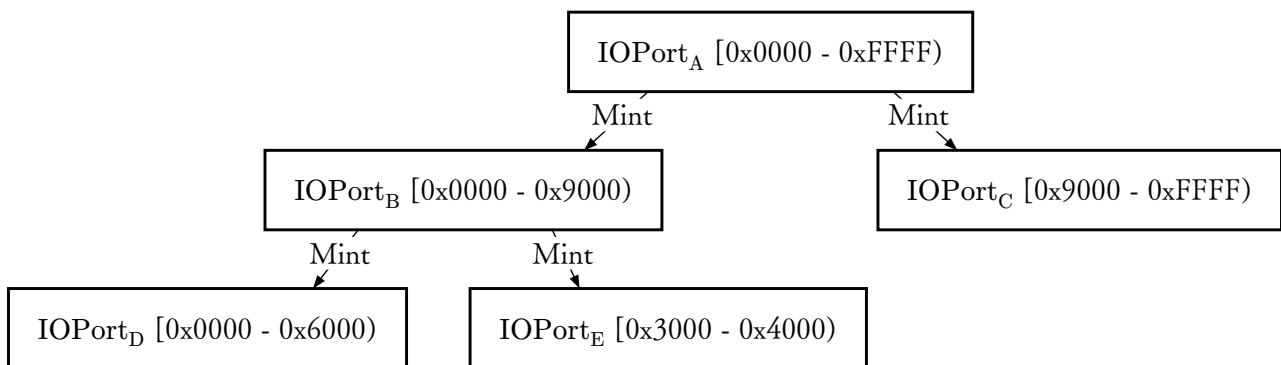


Figure 17: Mint による IO Port の Subset 生成例

4.1.21.2. Capability Call

read

IO Port から値を Read する。

type	name	description
capability_descriptor	io_port_descriptor	対象 IO Port への Descriptor

Table 44: read の引数

type	name	description
word	data	IO Port から Read した値

Table 45: read の戻り値

write

IO Port に値を Write する.

type	name	description
capability_descriptor	io_port_descriptor	対象 IO Port への Descriptor
word	data	IO Port に Write する 値

Table 46: write の引数

mint

IO Port の Subset を生成する.

type	name	description
capability_descriptor	io_port_descriptor	対象 IO Port への Descriptor
word	new_address_min	生成する Subset の最 低 Address
word	new_address_max	生成する Subset の最 高 Address
capability_descriptor	node_descriptor	生成した Subset を格 納する Node への Descriptor
word	index	生成した Subset を格 納する Node の Index (Offset)

Table 47: mint の引数

4.1.22. Virtual CPU Capability

Virtual CPU Capability は Virtualization の根幹をなす Capability である。Virtual Machine は Virtual CPU 上で動作する。したがって、Virtual CPU の状態を操作することで User-Level の Virtual Machine Monitor を実装する。

Virtual CPU には Capability Call として Enter 操作が定義される。ある Context が Enter を実行するとその Context は Virtual CPU に Bind され Schedule される。

4.1.22.1. Virtual CPU State Descriptor

Virtual CPU は Exit Reason の発生時に Virtual CPU に Enter している Context へ制御を返す。ここで Virtual CPU の状態を更新し、再び実行を開始することを考える。当然ながら状態を取得する必要がある、また書き込める必要がある。ここで一度制御が返されるたびに Get State のような操作を実行し、その後 Set State のような操作を実行しては非効率である。このコストを削減するために毎回すべての状態を転送する手法も考えられるが、これも不要な Copy を伴うため非効率である。

そこで Virtual CPU State Descriptor を導入する。この Descriptor は転送する State を選択するための Bit Flag であり、Virtual CPU の Exit Reason ごとに設定可能である。

4.1.22.2. Virtual IRQ

Virtual CPU は Virtual IRQ を Inject することができる。Inject された IRQ は Virtual Machine 中で実際の IRQ として扱われる。使用例として、例えば定期的に Virtual IRQ を Inject することで Clock Interrupt を Emulate することができる。また、他の Device Driver のような割り込みを使用する機構も Emulate できる。

4.1.22.3. Capability Call

現在正式に実装されているわけではなく、仕様のみが定義されている。

enter

Virtual CPU と Context を Bind し、Virtual Machine の実行を開始する。

type	name	description
capability_descriptor	target_descriptor	対象 Virtual CPU への Descriptor

Table 48: enter の引数

exit

実行中の Virtual Machine を外部から中断し制御を返す.

type	name	description
capability_descriptor	target_descriptor	対象 Virtual CPU への Descriptor

Table 49: exit の引数

read_state

Virtual CPU の状態を取得する.

type	name	description
capability_descriptor	target_descriptor	対象 Virtual CPU への Descriptor
word	state_descriptor	読み出す Field を示す State Descriptor

Table 50: read_state の引数

type	name	description
word[n]	fields	読み出した Field の値

Table 51: read_state の返回值

write_state

Virtual CPU の状態を書き込む.

type	name	description
capability_descriptor	process_control_block	対象 Process Control Block への Descriptor
word	state_descriptor	書き込む Field を示す State Descriptor
word[n]	fields	書き込む Field の値

Table 52: write_state の引数

inject_irq

Virtual IRQ を Inject する.

type	name	description
capability_descriptor	target_descriptor	対象 Virtual CPU への Descriptor
word	irq_number	Inject する IRQ Number

Table 53: inject_irq の引数

configure_address_space

Virtual CPU の Address Space を設定する. この Address Space は Host と Guest の Physical Address Space を Mapping する.

type	name	description
capability_descriptor	target_descriptor	対象 Virtual CPU への Descriptor
capability_descriptor	virtual_address_space	Bind する Virtual Address Space への Descriptor

Table 54: configure_address_space の引数

configure_state_descriptor

Virtual CPU State Descriptor を設定する.

type	name	description
capability_descriptor	target_descriptor	対象 Virtual CPU への Descriptor
word	exit_reason	State Descriptor を設定する Exit Reason
word	target_state_descriptor	Exit Reason に Link させる State Descriptor

Table 55: configure_state_descriptor の引数

inject_irq

Virtual IRQ を Inject する.

type	name	description
capability_descriptor	target_descriptor	対象 Virtual CPU への Descriptor
word	irq_number	Inject する IRQ Number

Table 56: inject_irq の引数

4.1.23. Virtual Address Space Capability

Virtual Address Space Capability は Virtual CPU の Physical Address Space を抽象化する Capability である。この Capability は Host Physical Address (HPA) と Guest Physical Address (GPA) の Mapping を行う。したがって、Guest Virtual Address は Virtual CPU 上の Guest 自身が設定する。

4.1.23.1. Capability Call

現在正式に実装されているわけではなく、仕様のみが定義されている。

map

Virtual Page Table や Frame を Guest Physical Address Space に Map する。

type	name	description
capability_descriptor	target_descriptor	対象 Virtual Address Space への Descriptor
capability_descriptor	memory_descriptor	対象に Map する Virtual Page Table もしくは Frame への Descriptor
virtual_address	address	Map する Physical Address
memory_attribute	attribute	Map に使用する属性

Table 57: map の引数

unmap

Virtual Page Table や Frame を Guest Physical Address Space から Unmap する。

type	name	description
capability_descriptor	target_descriptor	対象 Virtual Address Space への Descriptor
capability_descriptor	memory_descriptor	対象から Unmap する Virtual Page Table もしくは Frame への Descriptor
virtual_address	address	Unmap する Physical Address

Table 58: unmap の引数

get_unset_depth

Virtual Address Space に Physical Address を Map するうえで、まだ Map されていない Virtual Page Table の Depth を取得する。

type	name	description
capability_descriptor	target_descriptor	対象 Virtual Address Space への Descriptor

Table 59: get_unset_depth の引数

type	name	description
word	depth	Map されていない Virtual Page Table の Depth

Table 60: get_unset_depth の返り値

4.1.24. Virtual Page Table Capability

Page Table とは異なり, Virtual Address Space における Host Physical Address と Guest Physical Address の Mapping を行う. それ以外は Page Table と同等の構造を持つ.

4.1.24.1. Capability Call

現時点で Virtual Page Table Capability に Capability Call は存在しないが, 将来的にそれ自体の Depth を確認するための `get_depth` が追加される予定である.

4.1.25. ABI

A9N Microkernel の HAL は Architecture ごとに Kernel ABI を定義する。ここでいう ABI は、主に Kernel Call と Virtual Message Register における Hardware Register の Mapping を指す。

4.1.25.1. Hardware-Independent ABI Constants

Kernel Call Type は全ての ABI における共通値が以下として定義される (cf., Table 61) :

type	name	description
sword	Capability Call	-1
sword	Yield Call	-2
sword	Debug Call	-3

Table 61: Kernel Call Type

通常の System における Kernel Call Type は System Call Number として知られるが、A9N Microkernel ではこの値を負数として定義する。Kernel Call 時に存在しない Kernel Call Type の値を指定すると Illegal Kernel Call Fault が発生する。したがって、ABI-Level の Kernel Call (System Call) Emulation が実現しやすくなる。

4.1.25.2. x86_64

x86_64 における Kernel Call の ABI は以下のように定義される (cf., Table 62) :

name	description
Kernel Call Type	Mapped to RDI
Reserved	Mapped to RAX (破壊される)

Table 62: x86_64 における Kernel Call ABI

また、Virtual Message Register の ABI は以下のように定義される (cf., Table 63) :

name	description
Message Register [0]	Mapped to RSI
Message Register [1]	Mapped to RDX
Message Register [2]	Mapped to R8
Message Register [3]	Mapped to R9
Message Register [4]	Mapped to R10
Message Register [5]	Mapped to R12
Message Register [6]	Mapped to R13
Message Register [7]	Mapped to R14
Message Register [8]	Mapped to R15

Table 63: x86_64 における Virtual Message Register ABI

これを超える Message Register (e.g., Message Register [n , $n \in \mathbb{N}$, $n > 8$]) は IPC Buffer の Message Field [n] に Mapping される。

4.1.26. Boot Protocol

A9N Microkernel は A9N Boot Protocol をもって Boot する必要があり、この根幹をなすのが Boot Info である。Boot Info は Kernel の起動に必要な情報を格納する構造体であり、以下のように定義される (cf., Listing 9) :

```
static constexpr a9n::word ARCH_INFO_MAX = 8;

struct boot_info
{
    memory_info      boot_memory_info;
    init_image_info  boot_init_image_info;
    a9n::word        arch_info[ARCH_INFO_MAX];
} __attribute__((packed));
```

Listing 9: A9N Boot Info

memory_info

Memory Info(Listing 10) は Kernel に利用可能な Physical Memory Region を伝達するための構造である。Kernel の起動前にメモリ領域の開始 Address, Page Size を Unit とする Page Count, および Memory Type を収集し格納する必要がある。Kernel はこの情報をもとに Generic を生成し、後述する Init Info (Listing 13) へ情報を再格納して User に委譲する。


```

enum class memory_map_type
{
    FREE,
    DEVICE,
    RESERVED,
};

struct memory_map_entry
{
    a9n::physical_address start_physical_address;
    a9n::word              page_count;
    memory_map_type        type;
};

struct memory_info
{
    a9n::word              memory_size;
    uint16_t               memory_map_count;
    memory_map_entry *memory_map;
};

```

Listing 10: A9N Boot Info における Memory Info

init_image_info

A9N Microkernel は ELF や PE といった特定の Executable Format に依存しない。したがって、Kernel が起動する前の Bootloader Phase において Init Server を適切に展開し Load する必要がある。そのようにして Load された Init Server の情報は Init Image Info (cf., Listing 11) に格納され Kernel に渡される。

```

struct init_image_info
{
    a9n::physical_address loaded_address;
    a9n::word              init_image_size;
    a9n::virtual_address  entry_point_address;
    a9n::virtual_address  init_info_address;
    a9n::virtual_address  init_ipc_buffer_address;
} __attribute__((packed));

```

Listing 11: A9N Boot Info における Init Image Info

arch_info

Architecture Info は Architecture-Specific な Field であり，主に HAL の起動に使用される．また，殆どの場合 Init Info (Listing 13) の同名 Field へ再格納される．

4.1.26.1. x86_64

x86_64 における Boot Info の Architecture Info は以下のように定義される．現在はたった一つの Field のみを使用されており，残りは将来の拡張用に予約されている (cf., Table 64) :

name	description
Architecture Info[0]	RSDP ¹⁶

Table 64: x86_64 における Architecture Info

現在の A9N (x86_64) は EDK2-Based の A9NLoader によって UEFI 環境上で Boot されるため，RSDP は A9NLoader が UEFI の手続きに従って取得する．

4.1.26.2. Jump to Kernel

Boot Info (cf., Listing 9) は Kernel Main Entry Point への引数として渡される必要がある．Kernel Main Entry Point は以下のように定義される (cf., Listing 12) :

¹⁶ACPI における RSDP:Root System Description Pointer の Physical Address.

```
extern "C" int kernel_entry(a9n::kernel::boot_info *target_boot_info);
```

Listing 12: Kernel Main Entry Point

この Entry Point は Architecture に依存しないものである。したがって、典型的に Kernel は以下の Phase によって起動される (cf., Figure 18) :

1. Bootloader が起動される。
2. Bootloader が Kernel と Init Server を Load する。
3. Bootloader が HAL の提供する Architecture-Specific な Entry Point へ Jump する。
4. HAL が Kernel Main Entry Point へ Jump する。
5. Kernel が Init Server を起動する。

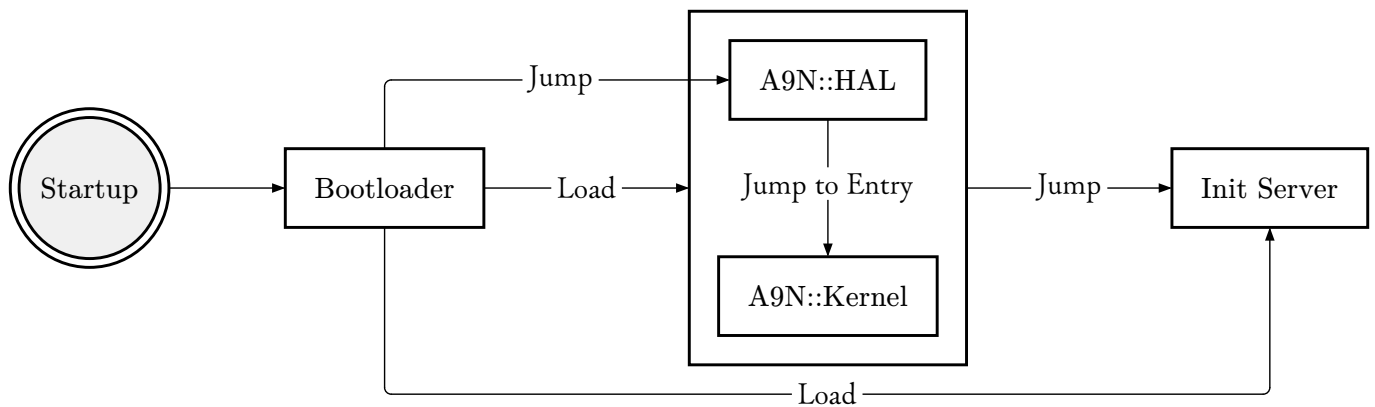


Figure 18: Kernel Boot Sequence

4.1.27. Init Protocol

A9N Microkernel は Init Server を Boot Info の情報をもとに生成し起動する。Init Server に利用可能な Capability や初期状態を提供するため、Init Info 構造体 (cf., Listing 13) が定義される。

```
struct init_info
{
    // kernel description
    a9n::word kernel_version {};

    // architectural information
    a9n::word arch_info[ARCH_INFO_MAX];

    // initial ipc buffer
    a9n::virtual_address      ipc_buffer;

    // initial generic
    generic_descriptor         generic_list[INITIAL_GENERIC_COUNT_MAX];
    a9n::word                  generic_list_count;
};
```

Listing 13: A9N Init Info

Generic Descriptor (cf., Listing 14) は Init Server が利用可能な Generic の情報を格納する構造体である。Generic Descriptor の Index はそのまま Init Slot Offset (cf., Table 65) で定義された Generic Node 内の Index となる。

```
struct generic_descriptor
{
    a9n::physical_address address;
    uint8_t                size_radix;
    bool                   is_device;
};
```

Listing 14: Generic Descriptor

Init Server に限り Kernel が初期状態として Root Node とその内容を提供する。Kernel が生成した Capability は以下の Init Slot Offset (cf., Table 65) に従い、Init Server の Root Node へ格納される：

name	description
Index[0]	Reserved
Index[1]	Process Control Block
Index[2]	Process Address Space
Index[3]	Process Root Node
Index[4]	Process Page Table Node
Index[5]	Process Frame Node
Index[6]	Process IPC Buffer Frame
Index[7]	Generic Node
Index[8]	Interrupt Region
Index[9]	IO Port

Table 65: Init Slot Offset

特筆すべきは Index[3] の Process Root Node である。これは格納されている Node 自体を表す再帰的な Capability である。Root Node に対する操作を直接実行することはできないが、このような手法を用いることですべてを操作可能とする。

Init Info の内容を書き込む Address は簡単な計算で求めることができる。A9N Microkernel は 0 を基準とした Virtual Address の使用を Init Server に要求するため、(Listing 11) の Field である LoadedAddress + InitInfoAddress¹⁷ が Init Info の Physical Address となる。

¹⁷Virtual Address は単なる Offset となる。

4.1.28. Build System

以前の A9N Microkernel は Makefile によって Bootloader や Init Server と統合して Build されていたが、正しく分割されておらず本質的に Architecture-Dependent であった。これを解決するため、現在は構造化された CMake によって Build されるように変更された。具体的な構成を以下に示す (cf., Figure 19) :

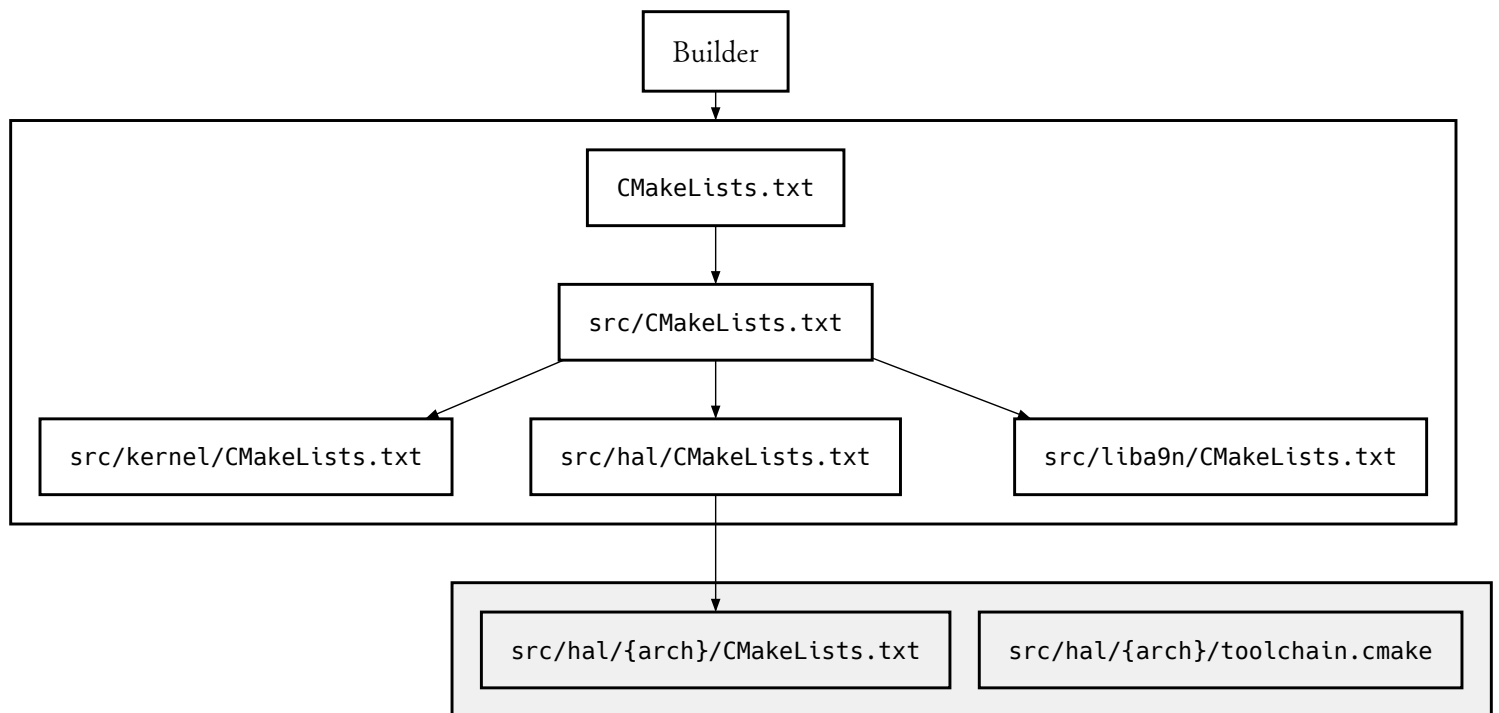


Figure 19: CMakeLists の構造

Source Code における Architecture-Dependent な部分の Build は `src/hal/{arch}/CMakeLists.txt` へ分離され、Toolchain-Specific な Toolchain File もまた `src/hal/{arch}/toolchain.cmake` へ分離された。これにより、構成する `CMakeLists.txt` の大半が Architecture-Independent となり、Porting が容易になった。

4.2. Nun Operating System Framework の開発

A9N Microkernel をコアとする OS を構築するためには、A9N Init Protocol (cf., Section 4.1.27) で示した Protocol に従って Init Server を構築する必要がある。これを容易かつ Secure に実現するため、Rust を用いて OS を構築可能な Nun Operating System Framework を開発した。

4.2.1. API Library

Nun API Library は A9N Microkernel における ABI 規約 (cf., Section 4.1.25) を満たした Kernel Call を実装する Library である、この Library は Rust の恩恵を最大限に活かした Error Handling と操作が可能であり、OS の Modern な開発を実現する。

4.2.1.1. Basic Types

A9N Microkernel における基本型を Rust における型として再定義する (cf., Table 66). そのため、機能的には C++ における実装と同等である。Rust の Coding Guideline に従い、C++ において Snake Case で表現されていた型は Camel Case で実現される。これらは Nun Module 内に存在し、`nun::*` のように使用する。

name	description
<code>Word</code>	Architecture-Specific な幅を持つ符号なし整数型
<code>Sword</code>	Architecture-Specific な幅を持つ符号付き整数型
<code>PhysicalAddress</code>	Physical Address を表現する型
<code>VirtualAddress</code>	Virtual Address を表現する型
<code>CapabilityDescriptor</code>	Capability Descriptor を表現する型

Table 66: Types

また、以下の定数群 (cf., Table 67) も同様に定義される。

name	description
BYTE_SIZE	Byte の Size
WORD_SIZE	`Word` の Size
PAGE_SIZE	Frame の Size
BYTE_BITS	1byte の Bit 数
WORD_BITS	`Word` の Bit 数 ; `WORD_SIZE` * `BYTE_BITS` として定義される

次に、Capability Error 型と Capability Result が定義される (cf., Listing 15). Kernel Call やそれに属する Capability Call は C ABI に依存しないため (cf., Section 4.1.9, Section 4.1.1), 標準の Result 型を使用することができる.

```
#[derive(Debug)]
#[repr(usize)]
pub enum CapabilityError {
    IllegalOperation,
    PermissionDenied,
    InvalidDescriptor,
    InvalidDepth,
    InvalidArgument,
    Fatal,
    DebugUnimplemented,
}

pub type CapabilityResult = result::Result<(), CapabilityError>;
```

Listing 15: Capability Result の Rust 実装

4.2.1.2. Kernel Call

Kernel Call における Capability Call, Yield Call, Debug Call は Rust の関数として Binding される.

4.2.2. Build System

Nun を用いた OS の Build には, Nun に統合された Cargo による Build System が使用される.

4.2.2.1. Custom Target

Nun には Build 対象とする Architecture 用の Custom Target が用意される. したがって, `cargo build --target {arch}-unknown-a9n18`により Build が可能となる. Custom Target によって Linker Script や Architecture-Dependent な Build Option が自動的に適用される. 現在, Target として `x86_64` が Support されている.

4.2.2.2. x86_64

`x86_64` における Custom Target は以下のように定義される (cf., Listing 16) :

¹⁸正確には
`cargo build --target Nun/arch/{arch}-unknown-a9n.json -Z build-std-features=compiler-builtins-mem --release`

```

{
  "llvm-target": "x86_64-unknown-none",
  "data-layout": "e-m:e-p270:32:32-p271:32:32-p272:64:64-i64:64-
i128:128-f80:128-n8:16:32:64-S128",
  "arch": "x86_64",
  "target-endian": "little",
  "target-pointer-width": "64",
  "target-c-int-width": "32",
  "os": "none",
  "executables": true,
  "linker-flavor": "ld.lld",
  "linker": "rust-lld",
  "relocation_model": "static",
  "pre-link-args": {
    "ld.lld": [
      "-T", "arch/x86_64/user.ld",
      "-Map=init.map",
      "-z", "norelro",
      "--static",
      "--nostdlib"
    ]
  },
  "panic-strategy": "abort",
  "disable-redzone": true,
  "features": ""
}

```

Listing 16: x86_64 における Custom Target

- Target Triple を LLVM の形式で指定する.
- その他 Target Field は Rust によって規定される形式 (cf., [15]) に沿い指定する.

4.2.3. Entry Point

Nun は Rust の Standard Library や Startup Runtime を提供するものではない. そのため, Startup に必要な Architecture-Dependent な処理と Architecture-Independent な処理を適切に分配しつつ Entry Point を定義する機構を実装した.

`nun::entry!`は Rust における宣言的 Macro であり, Listing 17 のように使用することで User-Defined な Entry Point を指定できる.

```
// <Entry Point>はメタ構文変数であり, 実際には`main`などの関数名が入る
nun::entry!(<Entry Point>);

<Entry Point>(init_info: &nun::InitInfo) {
    // nun::entry!によって指定されたEntry Pointにはinit_infoが渡される
}
```

Listing 17: User-Defined Entry Point

これは以下 (cf., Figure 20) に示す Flow で処理を実現する.

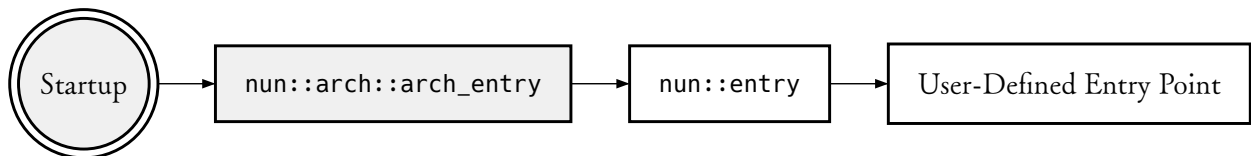


Figure 20: Startup Sequence

4.2.4. Code Exapmle

Nun を用いた最小な OS のコード例を付録に収録している (Section 9.3.1). わずか 8 行¹⁹で User Mode 上に動作する OS を実現している.

¹⁹コメント・空行を含まない.

4.3. KOITO の開発

KOITO は Nun (cf., Section 4.2) を用いて開発された A9N Microkernel をコアとする OS である。Rust を用いて Modern かつ Secure に開発されている。実態としては下図 (cf., Figure 21) で示すような、User-Level で動作する Server 群を 1 つとして捉えたものである。

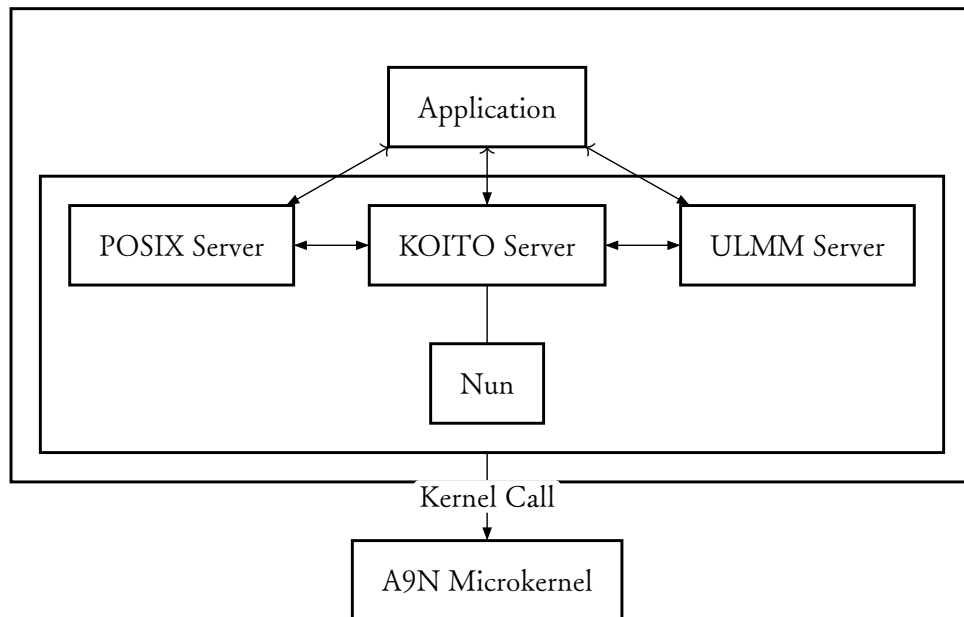


Figure 21: KOITO Architecture

4.3.0.1. ACI

KOITO は User Application との Interface として ACI (Application Capability Interface) を定義する。実行可能な Context ごとに Node の使用方法を定義することで、それぞれの User Application について共通の処理を実現する。現在の Version における ACI は以下のように定義される (cf., Listing 18) :

```

#[repr(usize)]
pub enum SlotOffset {
    // 0 is reserved : to prevent wrong specification by null descriptor
    Reserved = 0,
    RootNode = 1,
    ProcessControlBlock = 2,
    AddressSpace = 3,
    BufferFrame = 4,
    PageTables = 5,
    Frames = 6,
    OSPort = 7,
    Node = 8, // user-defined!
}

```

Listing 18: KOITO の定義する ACI

このうち、特に注目すべきは OSPort である。Process の作成時、Root Node の Offset: OSPort には KOITO Server と通信を行うための IPC Port が格納され、Identifier (cf., Section 4.1.17.4) として Process 固有の値が設定される。これは PID として KOITO Server によって使用される。

この OSPort を切り換え宛先を変更することで、本質的に OS そのものを切り替えることが可能となる。したがって、KOITO Server をもう一つ実行し隔離された状態で動作させることも可能である。他の活用例として、本来の通信先である KOITO Server の間に介在するような Proxy Server を実装し、完全に User-Level で OS-Level System Call を監視することも可能である。

4.3.1. KOITO Server

KOITO Server は所謂 Init Server であり、A9N Microkernel によって直接起動される。この Server は POSIX Server と ULMM Server、そして幾つかの Device Driver Server を起動し、その後 KOITO Shell を起動する。

KOITO Shell は OS として最初の動作を決定するために使用される。通常ここから POSIX Server を用いて User Application を実行するが、実験的機構を Test するための Debug Console の役割も果たす。

4.3.2. User-Level Memory Management Server

KOITO は A9N Microkernel の提供する Capability を用いることで, Secure に User-Level Memory Management を実現する. その根幹をなすのが ULMM Server である. 起動時, KOITO Server はほぼすべての Generic を ULMM Server に委譲する. ULMM Server それら Generic の Convert Policy を決定し, それに従って Memory Allocation を行う.

IPC によって Memory Allocation Request を受け取ると, ULMM Server は Convert Policy に従い Generic を Convert し, 生成した Capability の Transfer を実行する (cf., Section 4.1.17.5).

4.3.2.1. SLAB Allocator

ULMM Server は Request により SLAB として Kernel Object 用の Generic を Buddy Allocator (cf., Section 4.3.2.2) から Allocate する. 言い換えると, Kernel Object ごとに分割された Generic を Allocate する. これにより, Generic の特性を用いて (cf., Section 4.1.12.1) 半自動的に SLAB Allocator が実装される.

4.3.2.2. Buddy Allocator

ULMM Server は Buddy System を用いた Buddy Allocator を User-Level で実装する. Buddy Allocator は 2 の累乗をベースとし, 領域を再帰的に 2 分割していくことで Fragmentation を抑えつつ Allocation を実行するアルゴリズムである.

KOITO では, Capability を用いて Generic を $\frac{1}{2}$ サイズの子 Generic へ再帰的に分割していくことで実装される. ここで重要なのは, その Generic を必ず再利用の単位として扱うことである. A9N Microkernel において, Generic の再利用を行うにはそこから派生した Capability をすべて削除する必要がある (cf., Section 4.1.12.2). そのため, 割り当てる単位として Capability と Generic の Size を 1:1 で対応させることにより, Generic の結合処理を上位の Generic に対する Revoke 操作と同等とする.

4.3.3. POSIX Server

KOITO は限定的であるが POSIX API を提供するための POSIX Server を実装する。POSIX Server は基本的に `koito-libc` からの System Call Request を受け取り, それに対応する操作を実行するものである (cf., Figure 22).

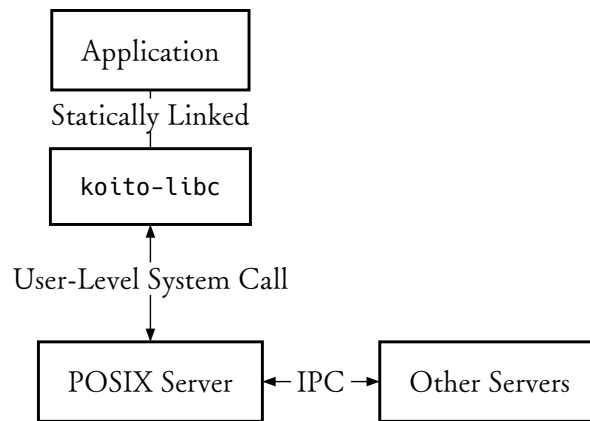


Figure 22: POSIX Server の略式 Architecture

POSIX Server は Process Management や Memory Management Frontend²⁰, 及び標準入出力などの機構を提供する。必ず POSIX Server のみで Request を完結させるわけではなく, 必要に応じて他の Server へ処理が委譲される。

現在, この POSIX Server には以下の System Call が定義されている (cf., Listing 19) :

²⁰Backend は ULMM Server である。

```
// same as linux syscall number for convenience
#[repr(usize)]
pub enum Syscall {
    Read = 0,
    Write = 1,
    Open = 2,
    Close = 3,
    Stat = 4,
    Fstat = 5,
    Lseek = 8,
    Brk = 12,
    Getpid = 39,
    Fork = 57,
    Execve = 59,
    Exit = 60,
    Wait = 61,
    Kill = 62,
    Link = 86,
    Unlink = 87,
    Times = 100,
}
```

Listing 19: POSIX Server の System Call Request 定義

4.3.3.1. Standard C Library

KOITO は Standard C Library として KOITO libc を提供しているが、現状における KOITO libc のベースとなるのは Newlib である。Newlib は Portable な Standard C Library であり、System Call の呼び出し部分を A9N と KOITO における IPC Call に置き換えることで機能要件を達成する。

Newlib は Makefile と Autoconf によって Build されるが、これを Modernize し CMake Integration を行った。この機構は OS に依存しないため、KOITO ではない他の System に適用したい場合も使用可能である。Porting に必要なものを以下に示す：

crt0

C Runtime. 通常、Stack Pointer や Environment Pointer を初期化し、引数を設定してから main へ Jump するために使用される。

linker.ld

User Application を C Runtime と Link するために必要な Linker Script.

newlib_glue.c

Newlib によって呼び出される, System Call を OS 固有の呼び出しに変換するための Glue Code.

4.4. liba9n の開発

通常、Kernel の開発においては C や C++ が選択される。しかし、これらの言語には安定性に欠かせない Error Handling 機構が不足している。まず、C においては単なる整数を Error 型とし、値 0 を成功として扱うことが一般的である。この手法には問題があり、関数が事実上ただの整数型を返さなければならず、何らかの値が必要な場合には Pointer による参照渡しを行う必要がある。というのも、-1 を Error として扱う場合、仮に -1 が正常値であったとしても Error として判定されてしまうからである。

では、Global な `errno` を用いるのはどうだろうか？ これも考えてみればすぐに問題が見えてくる。MultiProcessor 環境においてこの手法は危険である。したがって必ず Lock が必要となるが、Lock の取得には Overhead が存在するため、Performance-Critical な Kernel においては現実的でない。

C++ の例外機構を用いるのも、こと Kernel Heap を排除した Capability-Based Microkernel においては適切でない。例外機構には Dynamic な Memory Allocation が必要であり、暗黙的に Kernel Heap を要求する。また、失敗時の Unwinding にかかるコストも無視できない。

そこで、昨今の Modern な言語が備える機構を基礎とした、Freestanding C++20 用の Error Handling Library である `liba9n` を開発した。`liba9n` は Standard C++ Library に依存せず、また Dynamic な Memory Allocation を必要としない。したがって、Kernel Heap を持たず、なおかつ最小化したい Microkernel に適している。

4.4.1. `liba9n::std`

`liba9n::std` は `libc++` の Subset であり、Freestanding な環境のために再実装したものである。これには `liba9n::std::move`、`liba9n::std::forward` などの標準 Utility 関数や、`type_traits` といった Meta-Programming 機構が含まれる。また、よく使用される `std::array` や、実験的な Dynamic Memory Allocation を行わない `std::function` なども提供される。

4.4.2. `liba9n::option<T>`

`liba9n::option<T>`は、型を `Optional` とする —— つまり、共通の無効値を表現可能な型である。Haskell の `Maybe` や Rust の `Option<T>`, Standard C++ Library の `std::optional<T>`に相当する。

`liba9n::option<T>`は Monadic Operation を Support する。そのため、逐次処理を抽象化した高度な Error Handling が可能である。

4.4.2.1. Monadic Operation

and_then

値が存在する場合 (`None` でない場合) のみ `option<T>` の持つ値を Callback に渡し、実行結果を返す。このとき、Callback は `callback(T) -> option<T>` のような形をとる。値が存在しない場合は何も実行しない。

or_else

値が存在しない場合 (`None` の場合) のみ Callback の実行結果を返す。このとき、Callback は `callback(void) -> option<T>` のような形をとる。値が存在する場合は何も実行しない。

transform

値が存在する場合 (`None` でない場合) のみ Callback に渡し、実行結果を `Option<U>` で Wrap して返す。このとき、Callback は `callback(T) -> U` のような形をとる。値が存在しない場合は何も実行しない。

4.4.3. `liba9n::result<T, E>`

`liba9n::result<T, E>`は、型を `Result` とする —— つまり、成功時には成功値を、失敗時には Error 値を表現可能な型であり、Haskell の `Either` や Rust の `Result<T, E>`, Standard C++ Library の `std::variant<T, E>` もしくは `std::expected<T, E>` に相当する。

この型は `T` と `E` が異なる型であることを要請する。類似のものと比較して柔軟度はやや低くなるが、関数からの Return 時に型を推論させ、Helper Function を不要にすることができる。また、`liba9n::option<T>` と同様に Monadic Operation を Support する。

4.4.3.1. Monadic Operation

T を `ok` とし, E を `error` とする.

and_then

値が存在する場合のみ `ok` の値を `Callback` に渡し, 実行結果を返す. このとき, `Callback` は `callback(T) -> Result<U, E>` のような形をとる. 値が存在しない場合は何も実行しない.

or_else

値が存在しない場合のみ `error` の値を `Callback` に渡し, 実行結果を返す. このとき, `Callback` は `callback(E) -> Result<T, F>` のような形をとる. 値が存在する場合は何も実行しない.

transform

値が存在する場合のみ `ok` の値を `Callback` に渡し, 実行結果を `Result<U, E>` で `Wrap` して返す. このとき, `Callback` は `callback(T) -> U` のような形をとる. 値が存在しない場合は何も実行しない.

transform_error

値が存在しない場合のみ `error` の値を `Callback` に渡し, 実行結果を `Result<T, F>` で `Wrap` して返す. このとき, `Callback` は `callback(E) -> F` のような形をとる. 値が存在する場合は何も実行しない.

これらの Monadic Operation は Method Chaining による高度な逐次処理と Error Handling を実現する. この例を付録に収録している (Section 9.3.2).

4.4.4. Conditionally Trivial Special Member Functions

Kernel 内部で使用する型は高速であることが要求される. そのため, 可能な限り Memory ではなく Register を使用して値の受け渡しを実行したい. Itanium C++ ABI において, Trivial な型は Register に格納することが可能であると定められる [16]. そのため, `liba9n::option<T>` と `liba9n::result<T, E>` では Trivial 性を Member 型から引き継ぐために Conditionally Trivial Special Member Functions [17] を用いる. これにより, Trivial な型の場合は高速に扱うことが可能となる.

4.4.5. `liba9n::not_null<T>`

`liba9n::option<T>`や `liba9n::result<T, E>`には参照型を格納することができないが、`Pointer` 型によって参照を扱うと `Null Pointer` を許容してしまう。型レベルで参照が `Null` でないことを保証するために `liba9n::not_null<T>`を実装した。この型は参照を値として保持するが、作成時に必ず参照を必要とする。したがって、ある程度の安全性を保証することができる。

4.5. A9NLoader の開発

A9NLoader は A9N-Based な System (x86_64) を起動するための統合 Bootloader である。EDK2 を用いて開発されており、UEFI 環境で動作する。A9N Boot Protocol (cf., Section 4.1.26) に従って A9N Microkernel と Init Server を Load し、その後 Kernel に制御を移す。

4.5.1. CMake Integration

EDK2 は独自の Build System を持つが、これは CMake によって Wrap され Integration される。そのため、A9NLoader の Build Command は `mkdir build && cmake -B build && cmake --build build21` のように Simple となる。

²¹Out-of-Source Build

開発成果の特徴

5.1. HAL

A9N Microkernel は Hardware-Dependent な部分を分離するために HAL を導入し、高い Portability を達成する。そのため、A9N Microkernel における Kernel 部は完全に Hardware-Independent である。Kernel は HAL の Interface に対して操作を行い、HAL の各 Architecture における実装はその Interface を実装する。また、このような Hardware-Dependent 部分の分割は Source Code Level のみならず、Build System Level でも達成されている (cf., Section 4.1.28)

5.2. Capability

A9N Microkernel と KOITO は Capability を用いて Secure に User-Level Memory Management を実行する。それを用いることにより、KOITO は完全に User-Level の SLAB Allocator と Buddy Allocator を実現した。また、A9N Microkernel の持つ Capability API は Hardware-Independent である。したがって、簡単に Memory Management Server を他の Architecture に Porting できる。

5.3. 高速な IPC

A9N Microkernel は細かな最適化と Modern な Software Architecture により、極めて高速な IPC を実現した。以下に A9N Microkernel と既存 Microkernel の IPC Round-Trip Latency を示す (cf., Table 68).

Kernel	Architecture	Cycles
seL4 (Fastpath)	Intel® Core™ i7-6700	786
Fiasco.OC (Fastpath)	Intel® Core™ i7-6700K	2717
Zircon	Intel® Core™ i7-6700K	8157
A9N	Intel® N150	590

Table 68: Single-Core IPC Round-Trip Latency; cf., [18], [19]

Intel® N150@3.6GHz Processor において、A9N Microkernel は IPC Round-Trip Latency として 590Cycles (91.4ns) を達成した。この結果は x86_64 Architecture においてほぼ世界最速といえる値である。

5.4. Nun による OS 開発の簡易化

Nun Operating System Framework により、A9N Microkernel をコアとする OS の開発が極端に簡易化できる。これを証明するために DemoDay では Nun を用いて最小の OS を開発する Live Coding を実施し、およそ 2 分で Build から実行までを達成した。

5.5. 先行研究

3rd-Generation Microkernel の先行事例と比較し、A9N Microkernel の特徴を示す。

5.5.1. Zircon [1]

Zircon は Google によって開発された Little Kernel をベースとする Microkernel である。

5.5.1.1. Memory Management

Zircon は Handle²²を用いた User-Level Memory Management を実現しているが, Kernel によって Page Table の Management Policy を持つ [20].

5.5.1.2. Virtualization

Zircon は Hardware-Assisted Virtualization を Support し, Fuchsia と合わせて Type-2 Hypervisor としての機能を提供する [21].

5.5.1.3. Userland Software

Userland Software として Fuchsia という成熟した OS を持つ.

5.5.2. seL4 [2]

seL4 は NICTA の Trustworthy Systems によって開発された Capability-Based Microkernel である. 高速な IPC 機構を備え, かつ Formal Verification により Kernel の Correctness を証明している.

5.5.2.1. Memory Management

Kernel Heap を持たず, ほぼすべての Memory Management を Capability によって User-Level で行う. また, これらは最適化の観点からあえて Hardware-Dependent な Kernel-Level API を持つと明言されている [22].

5.5.2.2. Virtualization

seL4 は Hardware-Assisted Virtualization を Support し, VirtualizationType-1 Hypervisor としての機構を提供する. これらは Capability を用いて操作可能であり, 大部分の機能は User-Level VMM で実現される. ただし, Virtualization API の殆どが Hardware-Dependent であるため抽象度は低い. 例えば x86_64 の場合, Virtualization API は VMCS を操作するための薄い Wrapper に過ぎない.

²²Capability と同等の, Kernel Object を操作するために使用される Token のこと.

5.5.2.3. Userland Software

seL4 の Userland Software は特に成熟していないが, Nun と同様に CAmkES [23] という Component-Based Framework が提供される. また, 準標準となるような OS として LionsOS が開発されている.

5.5.3. A9N の優位点

5.5.3.1. Memory Management

A9N Microkernel は Kernel Heap を持たず, seL4 と同様に Capability を用いた User-Level Memory Managent を実現する. また, 基本的に Kernel-LevelAPI は Hardware-Independent であるため, 他の Architecture へ容易に Porting できる.

5.5.3.2. Virtualization

A9N Microkernel は Type-1 Hypervisor 機構を備え, Capability-Based Virtualization を実現する. また, Virtual Message Register の Layout 以外は Hardware-Independent な API であり, Porting や概念の理解が容易である.

今後の課題，展望

6.1. Document

A9N Microkernel の Document についてはある程度整備されているが，まだ不十分である．また，その他の Project(i.e., Nun, KOITO, A9NLoader, liba9n)については全く Document が存在しない．Document の不足は User の拡充における重大な障害となるため，優先して Document の整備を行っていく予定である．

6.2. Porting

HAL が存在し Kernel の分離は成されている一方で，まだ他 Architecture への Porting が行われていない．Test Code を実行するための HAL 実装は存在しているが，これは Porting とは言い難い．まずは RISC-V 64bit(RV64)への Porting を行い，その後 AArch64 への Porting を実施する予定である．

6.3. POSIX Compatibility

現在の POSIX Server は不完全であり，POSIX 準拠の Software を完全に移植できていない．現在の実装の根幹となる Newlib の POSIX 対応箇所は限定的であるため，POSIX-Compatible な移植しやすい実装である mlibc へ移行する予定である．

6.4. User-Level Virtual Machine Monitor

Virtualization Extension である各 Capability (i.e., Virtual CPU, Virtual Address Space, Virtual Page Table) についての実装はある程度進んでいる一方で未だ未完成である．というのも，何度も手戻りが発生し 3 回以上の API 再設計を実施したためである．ま

た，この不安定な Virtualization API の User である VMM の実装も当然ながら未完成である．ただし，現在 API 設計は現在ほぼ完了し成熟へ向かっている．また，Virtualization 機構を実際に使用する HAL 内の実装もほぼ完了している．したがって，数週間以内には簡易的な VMM の実装を完了し，Linux を動作させることを目指す．

6.5. Future

A9N Microkernel や Nun, KOITO の開発は未踏期間終了後も継続していき，既存 System の置換を中期的な目的とする．時間は必要だが，既存資産の再利用を POSIX Server や Virtualization Extension によって実現することにより，段階的な移行を Support していきたいと考えている．

現代の巨大な System を 1 人の力で実装することは難しい．だが，Microkernel のように小さい Kernel は 1 人でも大半を実装できる．もちろん OSS であり Community を形成して開発していくことも考えている．ただ，人海戦術に依存せずともに少人数も開発の継続が可能であり，またその意思も持っている．

普及後は Microkernel-Based な System を用いて計算機同士の Communication を無限に Scale-Out し，Ubiquitous Computing を実現していきたいと考える．

実施計画書内容との相違点

基本的には計画書と同等だが、計画していなかった Nun OS Framework と Hypervisor Extension の開発を実施した。Nun OS Framework は実装中盤の思いつきで生まれたものであり、計画書には記載されていない。ただし、Hypervisor Extension の実装は本プロジェクト期間の終了後に計画している旨が記載されている。

8

成長の自己分析

Project 開始時点では、既存の論文やコードを読みとき実装するスキルが不足していた。そのため、古く限られたリソースをもとに実装を行う以外の手法を持ち合わせていなかった。Project が進行するにつれ、先行事例の調査を実施する必要性が高まり最先端の研究に触れる機会が増えた。結果として論文やコードを読むスキルが向上し、新規実装の初手からそれを活かせるようになった。

また、複数の Layer を接続を考慮して実装したことで、上下それぞれに適した Protocol や Interface を設計できるようになった。

Software System を最下部から構築する経験は、通常の Software Development では得難い。理論だけでは通用せず、かといって実装だけでも行き詰まるために双方のバランスを取りつつ進める必要があった。

さらに、本 Project は低レイヤー領域の開発を 1 人で進める Project であり、多大な困難を伴った。しかし、このような—— ややスパルタともいえる環境は否応なく自己解決能力を鍛え、自立したエンジニアとしての成長を促進した。

結論として、この Project を通じ、System を基盤から実装できるエンジニアへと成長を遂げることができたように考える。

9

付録

9.1. 用語説明

Kernel

システムと Hardware を接続する中核をなすソフトウェア.

Monolithic Kernel

OS における多くの機構を内包する Kernel.

Microkernel

提供する機構を最小化した Kernel.

HAL

Hardware Abstraction Layer の略称. Kernel から直接 Hardware を呼び出すのではなく, 抽象 Interface による境界を設定することで移植容易性を高めるための仕組み.

OS

Kernel の提供する API を用い, 要件達成のための機構を User に提供するソフトウェア.

IPC

Inter-Process Communication の略称であり, ある Context と別の Context が相互に通信することを表す.

eBPF

Extended Berkeley Packet Filter の略称. Linux Kernel において, Kernel Space で動作するプログラムを実装し Kernel の動作を動的に変更するための仕組み.

Wasm

WebAssembly の略称. Portable かつ低レベルな Binary Code の共通表現を定義したもの.

TCB

Trusted Computing Base の略称であり, User が信頼しなければならない Hardware や Software の要素を表す.

PoLP

Principle of Least Privilege の略称であり, 最小特権の原則 [9] ともいう.

Policy/Mechanism Separation

機構と方針の分離 [8]. ある機構(Mechanism)が提供する機能とそれを利用する方針(Policy)を分離することで, 柔軟にシステムの変更を容易にすること.

Linux

Linus Torvalds によって開発された Monolithic Kernel.

C++

Bjarne Stroustrup によって開発された Multi-Paradigm Programing Language. System Programming に幅広く使用される.

Rust

Mozilla によって開発された Multi-Paradigm Programing Language. Linear Type System により Memory Safety を保証するため, C や C++に変わる System Programming Language として注目されている.

L4

Jochen Liedtke によって開発された, 高速な IPC 性能を持つ 2nd-Generation Microkernel.

seL4

NICTA の Trustworthy Systems によって開発された 3rd-Generation Capability-Based Microkernel.

Fiasco.OC

L4 系列の 3rd-Generation Capability-Based Microkernel.

Zircon

Google によって開発された 3rd-Generation Microkernel.

UEFI

Unified Extensible Firmware Interface の略称であり, OS と Platform 間の Interface を定義する規格.

EDK2

TianoCore Project によって開発されている UEFI Application の開発環境. また, UEFI 仕様の Reference Implementation.

9.2. 関連 Web サイト

<https://github.com/horizon2038/A9N>

A9N Microkernel

<https://github.com/horizon2038/A9NLoader>

A9N Boot Protocol(x86_64) に従った Reference Bootloader 実装

<https://github.com/horizon2038/Nun>

A9N Microkernel 上で動作する OS を開発するための Rust 製 Framework

9.3. Sample Codes

9.3.1. Nun を用いた Hello World プログラム

9.3.1.1. hello-world/src/main.rs

```
#![no_std]
#![no_main]

// using Nun!
use nun;

// configure entry point
nun::entry!(main);

fn main(init_info: &nun::InitInfo) {
    nun::println!("Hello, Nun World!");

    loop {}
}
```

9.3.2. liba9n::result<T, E>を用いた Method Chain

9.3.2.1. src/kernel/process/process_manager.cpp

```
kernel_result process_manager::switch_to_idle(void)
{
    return a9n::hal::current_local_variable()
        .transform_error(convert_hal_to_kernel_error)
        .and_then(
            [&](cpu_local_variable *local_variable) -> kernel_result
            {
                local_variable->current_process = &idle_process;
                local_variable->is_idle          = true;

                return {};
            }
        );
}
```

参考文献

- [1] Google, “Zircon.” [Online]. Available: <https://fuchsia.dev/fuchsia-src/concepts/kernel>
- [2] T. Systems, “seL4.” [Online]. Available: <https://sel4.systems/>
- [3] G. E. Moore, “Cramming more components onto integrated circuits,” *Proceedings of the IEEE*, vol. 86, no. 1, pp. 82–85, 1965.
- [4] M. Larabel, “The Linux Kernel Enters 2020 At 27.8 Million Lines In Git But With Less Developers For 2019.” [Online]. Available: <https://www.phoronix.com/news/Linux-Git-Stats-EOY2019>
- [5] T. Claburn, “Mixing Rust and C in Linux likened to cancer by kernel maintainer.” [Online]. Available: https://www.theregister.com/2025/02/05/mixing_rust_and_c_linux/
- [6] M. M. Swift, M. Annamalai, B. N. Bershad, and H. M. Levy, “Recovering device drivers,” *ACM Trans. Comput. Syst.*, vol. 24, no. 4, pp. 333–360, Nov. 2006, doi: 10.1145/1189256.1189257.
- [7] A. Chou, J. Yang, B. Chelf, S. Hallem, and D. Engler, “An empirical study of operating systems errors,” *SIGOPS Oper. Syst. Rev.*, vol. 35, no. 5, pp. 73–88, Oct. 2001, doi: 10.1145/502059.502042.
- [8] R. Levin, E. Cohen, W. Corwin, F. Pollack, and W. Wulf, “Policy/mechanism separation in Hydra,” *SIGOPS Oper. Syst. Rev.*, vol. 9, no. 5, pp. 132–140, Nov. 1975, doi: 10.1145/1067629.806531.
- [9] J. H. Saltzer, “Protection and control of information sharing in multics,” *SIGOPS Oper. Syst. Rev.*, vol. 7, no. 4, p. 119, Jan. 1973, doi: 10.1145/957195.808059.
- [10] “syscalls(2) - Linux manual page.” [Online]. Available: <https://man7.org/linux/man-pages/man2/syscalls.2.html>
- [11] J. B. Dennis and E. C. Van Horn, “Programming semantics for multiprogrammed computations,” *Commun. ACM*, vol. 9, no. 3, pp. 143–155, Mar. 1966, doi: 10.1145/365230.365252.
- [12] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design patterns*. Boston, MA: Addison Wesley, 1994.

-
- [13] K. Elphinstone and G. Heiser, “From L3 to seL4 what have we learnt in 20 years of L4 microkernels?,” in Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles, in SOSP '13. Farmington, Pennsylvania: Association for Computing Machinery, 2013, pp. 133–150. doi: 10.1145/2517349.2522720.
 - [14] M. Warton, “Single Kernel Stack L4,” Sydney, Australia, 2005.
 - [15] “Struct TargetOptions.” [Online]. Available: https://doc.rust-lang.org/nightly/nightly-rustc/rustc_target/spec/struct.TargetOptions.html
 - [16] “Itanium C++ ABI.” [Online]. Available: <https://itanium-cxx-abi.github.io/cxx-abi/abi.html>
 - [17] B. Revzin and C. Carter, “Conditionally Trivial Special Member Functions.” [Online]. Available: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0848r1.html>
 - [18] Z. Mi, D. Li, Z. Yang, X. Wang, and H. Chen, “SkyBridge: Fast and Secure Inter-Process Communication for Microkernels,” in Proceedings of the Fourteenth EuroSys Conference 2019, in EuroSys '19. Dresden, Germany: Association for Computing Machinery, 2019. doi: 10.1145/3302424.3303946.
 - [19] “seL4 Performance.” [Online]. Available: <https://sel4.systems/About/Performance/>
 - [20] Google, “Virtual Memory Object.” [Online]. Available: https://fuchsia.dev/fuchsia-src/reference/kernel_objects/vm_object
 - [21] Google, “Virtualization Overview.” [Online]. Available: <https://fuchsia.dev/fuchsia-src/development/virtualization/overview>
 - [22] G. Heiser, “seL4 Design Principles.” [Online]. Available: <https://microkerneldude.org/2020/03/11/sel4-design-principles/>
 - [23] T. Systems, “CAmkES.” [Online]. Available: <https://docs.sel4.systems/projects/camkes/>